UNIVERSIDAD CÉSAR VALLEJO

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS

Sistema Web de Control de Inventario y Gestión de Almacén aplicando métricas de calidad ISO 25010 para CITY OFERTAS IMPORTADORA E.I.R.L.

Segundo avance académico 2026

Integrantes

Aquino Salcedo Maria Fernanda

Carrasco Chavez Yerry Ericson

Cortez Quito Abraham David

Zelada Chinga Eliana Jazmin

Torres Cavero Luis Augusto

Juarez Gabino Sandro David

Curso Calidad de Software

Docente Dr. Iván Michell Castillo Jiménez

Piura Perú

Corte técnico 28 de octubre de 2026

ÍNDICE

Presentación del segundo avance....3
CAPÍTULO 1 INFORMACIÓN GENERAL....4
1.1 Razón social....4
1.2 Dirección y ubicación geográfica....4
1.3 Interesados y área beneficiaria....5
CAPÍTULO 2 ASPECTOS GENERALES....6
2.1 Definición del problema....6
2.2 Definición de los objetivos....6
2.3 Product Goal y delimitación....7
CAPÍTULO 3 METODOLOGÍA DE DESARROLLO....8
3.1 Selección de metodología....8
3.2 Justificación de la metodología....8
3.3 Descripción de la metodología....8
3.4 Roles y responsabilidades....9
CAPÍTULO 4 PLAN DE CALIDAD....10
4.1 Estrategia de calidad....10
4.2 Herramientas y técnicas....11
4.3 Recursos necesarios....11
4.4 Riesgos y mitigaciones....12
4.5 Criterios de éxito....12
4.6 Plan de comunicación....12
4.7 Cronograma del proyecto....13
Programación de actividades por grupos de procesos PMI....14
CAPÍTULO 5 REQUISITOS Y ARQUITECTURA DEL SISTEMA....15
5.1 Especificación de requisitos....15
5.2 Arquitectura del sistema....25
CAPÍTULO 6 ARTEFACTOS DE PRUEBA....59
6.1 Casos de prueba....59
6.2 Resultados de pruebas....60
6.3 Métricas de calidad....61
6.4 Aseguramiento de la calidad en el proyecto....61
CAPÍTULO 7 MANUALES Y CÓDIGO FUENTE....63
7.1 Manual de usuario....63
7.2 Código fuente....63
CONCLUSIONES Y RECOMENDACIONES....64
BIBLIOGRAFÍA....65
ANEXOS....66

Presentación del segundo avance

Este informe documenta el diseño integral y el avance verificable del Sistema Web de Control de Inventario y Gestión de Almacén de CITY OFERTAS IMPORTADORA E.I.R.L. El corte técnico corresponde al 28 de octubre de 2026. El diseño comprende las 63 historias del Product Backlog, incluidas las funciones futuras; los resultados de prueba corresponden exclusivamente a la versión y al entorno revisados.

La revisión confirmó una base funcional de catálogo, inventario, operaciones, seguridad y reportes. La suite automatizada existente finalizó satisfactoriamente, pero la validación adicional encontró tres criterios no conformes. Por ello, este documento presenta un segundo avance con evidencia técnica, sin declarar aceptación empresarial, puesta en producción ni cierre del proyecto.

El archivo Product_Backlog_WMS_Segundo_Avance.xlsx complementa el informe. Mantiene las historias, estimaciones y planificación del archivo de origen, amplía la aceptación a 191 escenarios y relaciona cada escenario con entradas, pasos, componentes y pruebas. El plan de prácticas y la guía docente orientan la organización de las actividades y sus evidencias mediante los grupos de procesos PMI y el marco Scrum.

CAPÍTULO 1 INFORMACIÓN GENERAL

1.1 Razón social

CITY OFERTAS IMPORTADORA E.I.R.L., con RUC 20611519886, es la empresa beneficiaria del proyecto. Su actividad comercial comprende la comercialización de artículos de bazar, librería, juguetería, menaje y productos del hogar. La información empresarial corresponde a los datos registrados en el expediente del proyecto.

1.2 Dirección y ubicación geográfica

La referencia proporcionada sitúa el establecimiento en la avenida Grau, cuadra 5, zona del Óvalo Grau, Piura. El documento de origen menciona comercios cercanos como referencias de ubicación. No se dispone de una ficha de domicilio legal ni de coordenadas verificadas; por tanto, no se presenta un mapa georreferenciado con una precisión que la evidencia disponible no permite sostener.



Figura 1 Frontis proporcionado en el informe de origen. Ubicación de referencia pendiente de verificación documental.

1.3 Interesados y área beneficiaria

Interesado	Necesidad o responsabilidad
Gerencia	Autorizar alcance y prioridades y revisar información de gestión
Almacén	Registrar recepciones, traslados, conteos y disponibilidad por ubicación
Compras	Consultar alertas y reposición y gestionar información de proveedores
Supervisión y contabilidad	Revisar ajustes, trazabilidad y valoración de inventario
Equipo académico	Diseñar, desarrollar, probar y documentar el sistema

La asignación de roles de aplicación debe ser aprobada por la empresa. Las responsabilidades académicas de Product Owner y Scrum Master no equivalen automáticamente a la representación legal ni a la autorización operativa del negocio.

CAPÍTULO 2 ASPECTOS GENERALES

2.1 Definición del problema

2.1.1 Descripción del problema

El diagnóstico preliminar identifica registros semimanuales y hojas de cálculo no integradas, diferencias entre existencias físicas y digitales, dificultades para localizar productos y ausencia de avisos oportunos de reposición. Estas situaciones justifican la centralización del inventario y el registro trazable de sus movimientos. La línea base de pérdidas, frecuencia de incidencias y tiempos de búsqueda aún debe cuantificarse para evaluar posteriormente la mejora operativa.

Un registro de inventario confiable requiere controlar tanto el movimiento como su contexto: producto, cantidad, costo, motivo, documento, usuario y ubicación. También debe evitar descuentos incompatibles con el saldo disponible y reintentos que dupliquen una operación. El sistema aborda estas necesidades mediante un ledger de movimientos, saldos por ubicación, transacciones y controles de acceso.

2.1.2 Formulación del problema general

¿De qué manera el desarrollo y la evaluación de un sistema web de inventario y almacén, mediante criterios de calidad seleccionados de ISO/IEC 25010, puede mejorar la gestión y trazabilidad de la mercadería de CITY OFERTAS IMPORTADORA E.I.R.L.?

2.1.3 Problemas específicos

Se requiere determinar cómo representar las reglas del negocio en historias y escenarios verificables; cómo preservar la integridad de movimientos, costos y saldos; y cómo comprobar la calidad funcional, mantenibilidad, rendimiento e interacción con usuarios mediante evidencia reproducible. La mejora operativa deberá comprobarse posteriormente comparando una línea base con resultados de uso en la empresa.

2.2 Definición de los objetivos

2.2.1 Objetivo general

Desarrollar un sistema web de control de inventario y gestión de almacén para CITY OFERTAS IMPORTADORA E.I.R.L. y evaluar sus atributos de calidad mediante criterios y métricas definidos a partir de ISO/IEC 25010.

2.2.2 Objetivos específicos

Implementar la gestión de productos, ubicaciones, proveedores y políticas de stock. Registrar recepciones, transferencias, ajustes, ventas importadas y conteos con integridad transaccional y trazabilidad. Diseñar y ejecutar pruebas unitarias, de integración y de aceptación con datos normales, negativos y de frontera. Evaluar mantenibilidad e integridad con herramientas automáticas y planificar mediciones de rendimiento e interacción con usuarios antes de la entrega final.

2.3 Product Goal y delimitación

El Product Goal propuesto es disponer de un inventario centralizado y trazable que permita conocer las existencias por ubicación, registrar operaciones sin duplicidad y apoyar la reposición mediante alertas y reportes. El backlog contiene 63 historias; 60 pertenecen a la fase inicial y tres están registradas como evolución futura: notificaciones externas HU-047, operación sin conexión HU-055 y captura móvil optimizada HU-056.

El diseño objetivo incluye las 63 historias. El compromiso temporal de la fase inicial se mantiene entre el 19 de octubre y el 10 de diciembre de 2026, sujeto a la revisión del equipo y la empresa. La integración de ventas consiste en importar operaciones del POS; no se acredita un sistema de facturación electrónica, contabilidad completa, tienda virtual ni procesamiento de pagos.

CAPÍTULO 3 METODOLOGÍA DE DESARROLLO

3.1 Selección de metodología

La gestión del proyecto se organiza con los grupos de procesos de Inicio, Planificación, Ejecución, Monitoreo y Control y Cierre utilizados en el plan de prácticas. Scrum orienta la construcción e inspección de los incrementos. Esta combinación permite relacionar el problema, el alcance, los riesgos y el cierre con la ejecución iterativa del software, conforme a la orientación de la guía docente suministrada.

3.2 Justificación de la metodología

Las reglas de stock, permisos y operaciones requieren revisión frecuente con quienes utilizarán el sistema. Los incrementos permiten presentar comportamientos observables y ajustar criterios antes de ampliar el alcance. La priorización debe atender el valor para la empresa y el riesgo de integridad: primero se verifica que el inventario no se corrompa; luego se amplían canales, automatizaciones e interfaces.

3.3 Descripción de la metodología

3.3.1 Artefactos y eventos Scrum

El Product Backlog reúne historias y prioridades y se vincula con el Product Goal. El Sprint Backlog reúne el objetivo y el trabajo seleccionado por los Developers. Un incremento solo puede considerarse terminado cuando satisface la Definition of Done acordada. El estado técnico del código no reemplaza la aceptación del escenario ni demuestra que se haya celebrado una ceremonia.

Se planifican Sprint Planning al inicio de cada iteración, Daily Scrum de 15 minutos para los Developers, Sprint Review con los interesados y Sprint Retrospective para mejorar el trabajo. Al corte, los registros de ejecución de estos eventos se mantienen pendientes de consolidación. El Excel distingue las fechas previstas de la evidencia disponible y conserva vacía la serie real del burndown, porque no existen registros diarios de esfuerzo restante.

3.3.2 Alcance y calendario por sprints

Sprint	Ventana prevista	Revisión prevista	Control del compromiso
1	19 octubre a 1 noviembre	30 octubre	Selección y tareas identificadas por sprint en el Excel
2	2 a 15 noviembre	13 noviembre	Refinamiento de aceptación y dependencias antes del inicio
3	16 a 29 noviembre	27 noviembre	Integración y verificación del incremento seleccionado
4	30 noviembre a 10 diciembre	10 diciembre	Consolidación, validación y entrega prevista

La distribución funcional por sprint se consulta en las columnas de planificación del backlog y en la hoja Sprints, evitando duplicar una asignación distinta en el informe. Los tres primeros intervalos duran 14 días; el último dura 11. Esta diferencia del archivo de origen

requiere ratificar la cadencia o tratar el último periodo como ventana de cierre. No se afirma una duración uniforme de dos semanas para los cuatro periodos.

3.4 Roles y responsabilidades

Integrante	Responsabilidad académica consignada
Aquino Salcedo Maria Fernanda	Product Owner y coordinación de prioridades
Carrasco Chavez Yerry Ericson	Scrum Master y facilitación del proceso
Cortez Quito Abraham David	Desarrollo del backend y arquitectura
Zelada Chinga Eliana Jazmin	Desarrollo de interfaz de usuario
Torres Cavero Luis Augusto	Modelamiento y gestión de base de datos
Juarez Gabino Sandro David	Pruebas y evaluación de calidad

Esta distribución define las responsabilidades académicas del equipo. La aprobación del alcance y de los resultados operativos exige la participación de un representante de la empresa. Las tareas del Excel que aún figuran por asignar deben recibir un responsable nominal; no se deduce esa asignación a partir de los autores de este informe.

CAPÍTULO 4 PLAN DE CALIDAD

4.1 Estrategia de calidad

4.1.1 Modelo de calidad

Se adopta ISO/IEC 25010:2023 como referencia para seleccionar atributos y organizar su evaluación. La edición 2023 presenta un modelo de nueve características; este proyecto evalúa un subconjunto pertinente y no declara certificación ni conformidad integral con la norma. La interacción con usuarios se trata bajo capacidad de interacción, conservando el término usabilidad cuando se describe el instrumento SUS.

La adecuación funcional se relaciona con las historias y escenarios; la fiabilidad con atomicidad, recuperación e integridad; la seguridad con autorización y auditoría; la mantenibilidad con pruebas, análisis estático y dependencias; y la eficiencia de rendimiento con mediciones controladas de operaciones críticas. La adaptación a dispositivos y entornos se verificará al completar las funciones móviles y la instalación reproducible.

4.1.2 Objetivos de calidad

Indicador	Meta del plan	Situación al corte
Cobertura de líneas ejecutables	Al menos 80 por ciento en el ámbito definido	95,79 por ciento en app/Modules
Éxito de pruebas antes del despliegue	Al menos 95 por ciento sobre el conjunto acordado	Suite existente 100 por ciento; aceptación específica 93,33 por ciento de 45 casos ejecutados
Tiempo medio de respuesta	Como máximo 1,5 segundos	No medido bajo un protocolo de carga
Puntuación SUS	Al menos 85 puntos sobre 100	No aplicada a usuarios de la empresa

El resultado SUS se expresa en puntos, no como porcentaje de usuarios satisfechos. Cada indicador exige identificar su población y denominador; la cobertura del dominio no mide la cobertura de interfaz y el éxito de una suite no equivale a la aceptación de todo el backlog.

4.1.3 Aseguramiento de la calidad

Cada historia debe tener criterios verificables, entradas y resultados esperados. Las revisiones de código deben atender reglas de negocio, límites transaccionales, autorización, precisión decimal y trazabilidad. Se mantienen pruebas automáticas y controles de formato, tipos y dependencias como controles preventivos. La revisión por pares y su registro se establecen como práctica requerida; no se presentan como revisiones históricas ya acreditadas.

4.1.4 Calidad durante el ciclo de vida y Definition of Done

La Definition of Done propuesta requiere criterios aplicables conformes; pruebas de regresión satisfactorias; ausencia de defectos de integridad o autorización sin resolver; aprobación de Pint, PHPStan y Deptrac; migraciones y dependencias reproducibles; documentación actualizada y demostración del flujo en el entorno acordado. Las métricas de

rendimiento, SUS y recuperación se incorporan cuando correspondan al incremento. Su aprobación debe ser registrada por los responsables del proyecto.

4.2 Herramientas y técnicas

4.2.1 Herramientas de desarrollo

El entorno revisado utiliza PHP 8.4.26 en Docker, Laravel 13.34.0, Filament 5.9.0, Livewire 4.4.7 y PostgreSQL 17. Vite y Tailwind participan en los recursos del panel. Las versiones instaladas se obtuvieron del entorno y de vendor; Filament y Livewire no aparecen declarados en composer.json ni composer.lock, lo que exige corregir la reproducibilidad de una instalación limpia. El panel se ejecuta dentro de la aplicación Laravel y utiliza las rutas web y los recursos existentes del proyecto.

4.2.2 Herramientas de prueba

Pest ejecuta pruebas unitarias y de integración. PCOV permite medir cobertura de líneas del ámbito configurado. Laravel Pint verifica estilo; PHPStan, tipos y errores estáticos; Deptrac, reglas de dependencias. JMeter, OWASP ZAP e Infection se contemplan para rendimiento, seguridad dinámica y mutación, pero no se registran resultados de esas herramientas en este corte.

4.2.3 Herramientas de gestión y modelamiento

El libro Excel concentra la planificación y la trazabilidad del proyecto. Enterprise Architect contiene los modelos editables de requisitos, clases, datos y arquitectura. ERwin se utiliza para el inventario lógico de entidades y atributos; la integridad del modelo relacional se documenta además en Enterprise Architect y en el diccionario de datos. La ejecución remota de integración continua se mantiene pendiente de verificación; la evidencia disponible corresponde a los controles locales en Docker.

4.2.4 Técnicas de calidad

Se combinan particiones de equivalencia, valores de frontera, escenarios negativos y comprobación de reglas de dominio. La integración prueba transacciones, restricciones, persistencia e idempotencia. Las pruebas de panel con Livewire o Filament no sustituyen una sesión completa en navegador ni una prueba de aceptación por personal de la empresa. La cobertura de ramas y caminos requiere instrumentación adicional y no se deduce de la cobertura de líneas.

4.3 Recursos necesarios

Participan los seis integrantes, los usuarios por rol y un representante de la empresa. Se requiere un equipo compatible con Docker y virtualización, memoria suficiente para la aplicación y PostgreSQL, un navegador y espacio para modelos y evidencias. La configuración de hardware del entorno de medición deberá registrarse antes de las pruebas de carga; no se declara como requisito validado una capacidad mínima de hardware sin mediciones.

4.4 Riesgos y mitigaciones

Riesgo	Impacto	Mitigación y responsable por rol
Reintentos con contenido distinto o doble apertura	Alto	Validar identidad completa y unicidad de apertura; desarrollo y QA
Permisos de exportación inconsistentes	Alto	Autorizar acción y servidor; desarrollo y QA
Instalación no reproducible	Alto	Declarar dependencias y verificar instalación limpia; desarrollo
Cambios de alcance o roles de negocio	Alto	Registrar decisiones y ratificar backlog; Product Owner y empresa
Pérdida de información	Alto	Copia externa y restauración comprobada; base de datos y operación
Rendimiento o interacción insuficientes	Medio	Medir carga y aplicar SUS antes de liberar; QA y usuarios
Cierre sin evidencia de aceptación	Alto	Revisar DoD, escenarios y acta; Product Owner y empresa

4.5 Criterios de éxito

El proyecto concluirá con las funciones comprometidas aceptadas por los responsables, las incidencias críticas resueltas, las metas de calidad comprobadas y la transferencia documentada. La exactitud del inventario se evaluará mediante conciliación física y digital en un periodo definido. No se promete eliminar todos los descuadres ni se atribuye una mejora porcentual sin medición antes y después.

4.6 Plan de comunicación

Actividad	Participantes	Frecuencia prevista	Registro requerido
Sprint Planning	Scrum Team	Inicio de sprint	Objetivo y selección de trabajo
Daily Scrum	Developers	Días de trabajo	Impedimentos y ajustes del plan según necesidad
Sprint Review	Equipo e interesados	Cierre de sprint	Feedback y decisiones del backlog
Retrospectiva	Scrum Team	Cierre de sprint	Acciones de mejora y seguimiento
Reporte de defecto	Desarrollo y QA	Al detectarse	Entrada, resultado, evidencia y revalidación
Validación y entrega	Empresa y equipo	Hitos acordados	Conformidad, manuales y pendientes

4.7 Cronograma del proyecto

El cronograma se organiza por grupos de procesos PMI. La ejecución de Scrum y el control se superponen durante el desarrollo; los grupos de procesos no se convierten en fases de software estrictamente consecutivas. Las marcas del cronograma representan trabajo previsto y no evidencia de reuniones o entregas realizadas.

Grupo	Periodo previsto	Actividades y entregables
Inicio	19 a 23 octubre	Necesidad, interesados, visión y documento de inicio por validar
Planificación	19 a 30 octubre y refinamiento continuo	Alcance, Product Goal, historias, aceptación, riesgos y calidad
Ejecución	19 octubre a 10 diciembre	Cuatro iteraciones, diseño, implementación, integración y documentación
Monitoreo y Control	19 octubre a 10 diciembre	Seguimiento, aceptación, riesgos, revisiones y actualización del backlog
Cierre	1 a 10 diciembre	Validación final, transferencia, manuales, recuperación y lecciones

La hoja Gestión PMI desarrolla doce actividades con fechas previstas, responsables por rol y estado de evidencia. Las hojas Cronograma y Sprints conservan la planificación original. Las revisiones previstas para el 30 de octubre, 13 y 27 de noviembre y 10 de diciembre son futuras respecto del corte; no se presentan como actas celebradas.

Programación de actividades por grupos de procesos PMI

Cada X indica una semana programada. Las actividades y fechas proceden de la hoja Gestión PMI; no acreditan reuniones ni entregas ya realizadas.

Grupo PMI	Actividad	1 19/10	2 26/10	3 02/11	4 09/11	5 16/11	6 23/11	7 30/11	8 07/12
Inicio	Identificar necesidad e interesados	X							
Planificación	Definir alcance y Product Goal	X	X						
Planificación	Refinar historias y aceptación	X	X	X	X	X	X	X	X
Planificación	Planificar sprints y calidad	X	X						
Ejecución	Sprint Planning al inicio de cada sprint	X		X		X		X	
Ejecución	Diseñar e implementar incrementos	X	X	X	X	X	X	X	X
Ejecución	Ejecutar pruebas e integrar	X	X	X	X	X	X	X	X
Monitoreo y Control	Inspeccionar avance y riesgos	X	X	X	X	X	X	X	X
Monitoreo y Control	Verificar aceptación y DoD	X	X	X	X	X	X	X	X
Monitoreo y Control	Review y Retrospective de cada sprint		X		X		X		X
Cierre	Validar y transferir el producto							X	X
Cierre	Registrar lecciones y pendientes								X

CAPÍTULO 5 REQUISITOS Y ARQUITECTURA DEL SISTEMA

5.1 Especificación de requisitos

5.1.1 Descripción del sistema

5.1.1.1 Propósito del sistema

El WMS centraliza el catálogo y las existencias de la empresa y registra los hechos que modifican el inventario. Las personas acceden por roles; los movimientos conservan su documento y ubicación; las consultas permiten revisar kardex, disponibilidad, alertas y valoración. La arquitectura objetivo añade los importadores pendientes, la conciliación programada, los canales externos y el cliente móvil definidos en el backlog.

5.1.1.2 Alcance del sistema

El alcance completo se distribuye en accesos y seguridad, catálogo, inventario, operaciones de almacén, conteos, ventas del POS, ubicaciones, proveedores, alertas y reportes, carga inicial, operación móvil y calidad de entrega. Los diagramas representan ese producto completo. La etiqueta propuesto identifica contratos y componentes aún no construidos; una figura de diseño no acredita su implementación.

Las 63 historias mantienen sus identificadores del archivo original. La matriz de aceptación conserva los 36 escenarios iniciales y amplía la especificación a 191. Cada fila permite seguir la cadena historia, escenario, entrada, pasos, clase o componente, prueba relacionada y resultado observado. Los casos sin ejecución específica no se marcan como aprobados por la sola existencia de código o de una prueba similar.

5.1.2 Requerimientos funcionales

Grupo funcional	Historias	Comportamiento objetivo
Accesos y seguridad	HU-001 a HU-007	Cuentas, autenticación, roles, contraseñas y auditoría
Catálogo e identificación	HU-008 a HU-016	Productos, SKU, categorías, unidades, códigos y políticas
Núcleo de inventario	HU-017 a HU-023	Ledger, saldo no negativo, idempotencia, costos, traslado, kardex y conciliación
Operaciones de almacén	HU-024 a HU-031	Recepciones, transferencias y ajustes con control y trazabilidad
Conteos y POS	HU-032 a HU-039	Conteo, reconteo, cierre, exactitud e importación de ventas
Ubicaciones y proveedores	HU-040 a HU-043	Generación de ubicaciones, estados, proveedores y asociaciones
Alertas y reportes	HU-044 a HU-051	Reposición, avisos, indicadores y exportaciones
Carga y movilidad	HU-052 a HU-056	Importación de catálogo, apertura y captura móvil, incluida evolución sin conexión

Grupo funcional	Historias	Comportamiento objetivo
Calidad y entrega	HU-057 a HU-063	Arquitectura, pruebas, automatización, continuidad, publicación y evaluación

Las HU-057 y HU-058 y las reglas de HU-017 a HU-021 incluyen restricciones o requisitos de ingeniería. Se representan como requisitos cuando no corresponden a una interacción independiente de un usuario. No se fuerza una equivalencia entre cada historia y un caso de uso de interfaz.

Existen diferencias de rol entre el backlog inicial y las políticas localizadas: el alta y edición del catálogo se restringen a Supervisor o Administrador, mientras algunas historias de origen mencionaban Almacenero o Compras. La propuesta del Excel alinea la descripción con la política observada y registra que necesita ratificación de negocio. No se considera una aprobación automática de esa modificación del requisito.

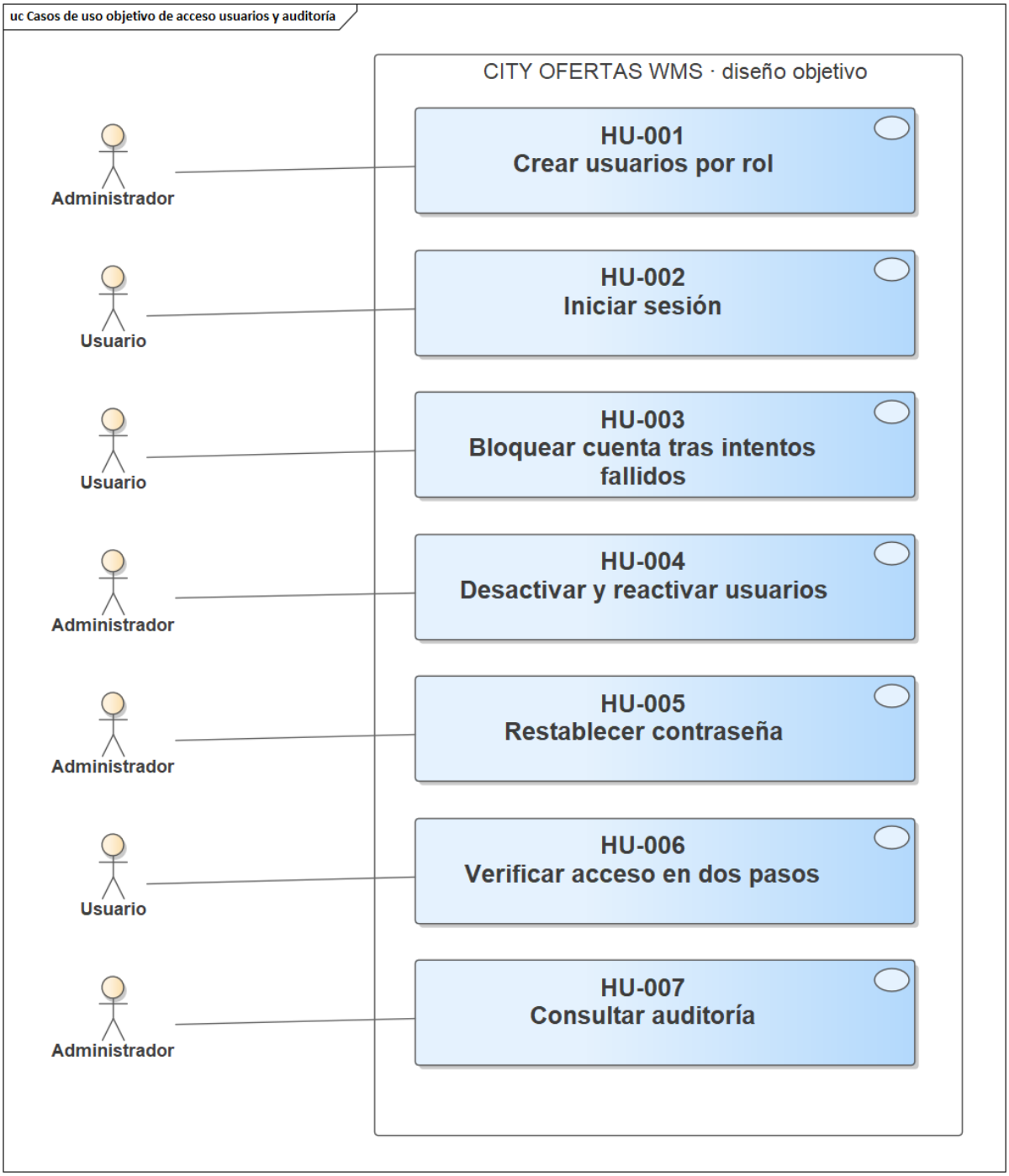
5.1.3 Requerimientos no funcionales

Código	Requisito verificable	Evidencia o validación prevista
RNF-01	Mantener exactitud decimal de cantidades y costos	Cantidades con tres decimales y costos con cuatro; BCMath y pruebas de frontera
RNF-02	Evitar saldos negativos y cierres parciales	Restricciones SQL, bloqueos y pruebas transaccionales
RNF-03	Restringir operaciones y exportación por permisos	Matriz de roles y pruebas de autorización en servidor e interfaz
RNF-04	Conservar trazabilidad del inventario	Ledger sin UPDATE, DELETE ni TRUNCATE y auditoría pertinente
RNF-05	Facilitar mantenibilidad	Análisis estático, contratos de dominio y suite de regresión
RNF-06	Instalar y operar de forma reproducible	Docker Compose, dependencias declaradas y ensayo de instalación limpia
RNF-07	Cumplir respuesta media de hasta 1,5 segundos	Prueba de carga con volumen, concurrencia y entorno documentados
RNF-08	Alcanzar SUS de al menos 85 puntos	Instrumento y protocolo con usuarios de la empresa
RNF-09	Recuperar información ante fallos	Política de respaldo y ensayo de restauración aún pendientes

5.1.4 Modelo de interacciones del producto completo

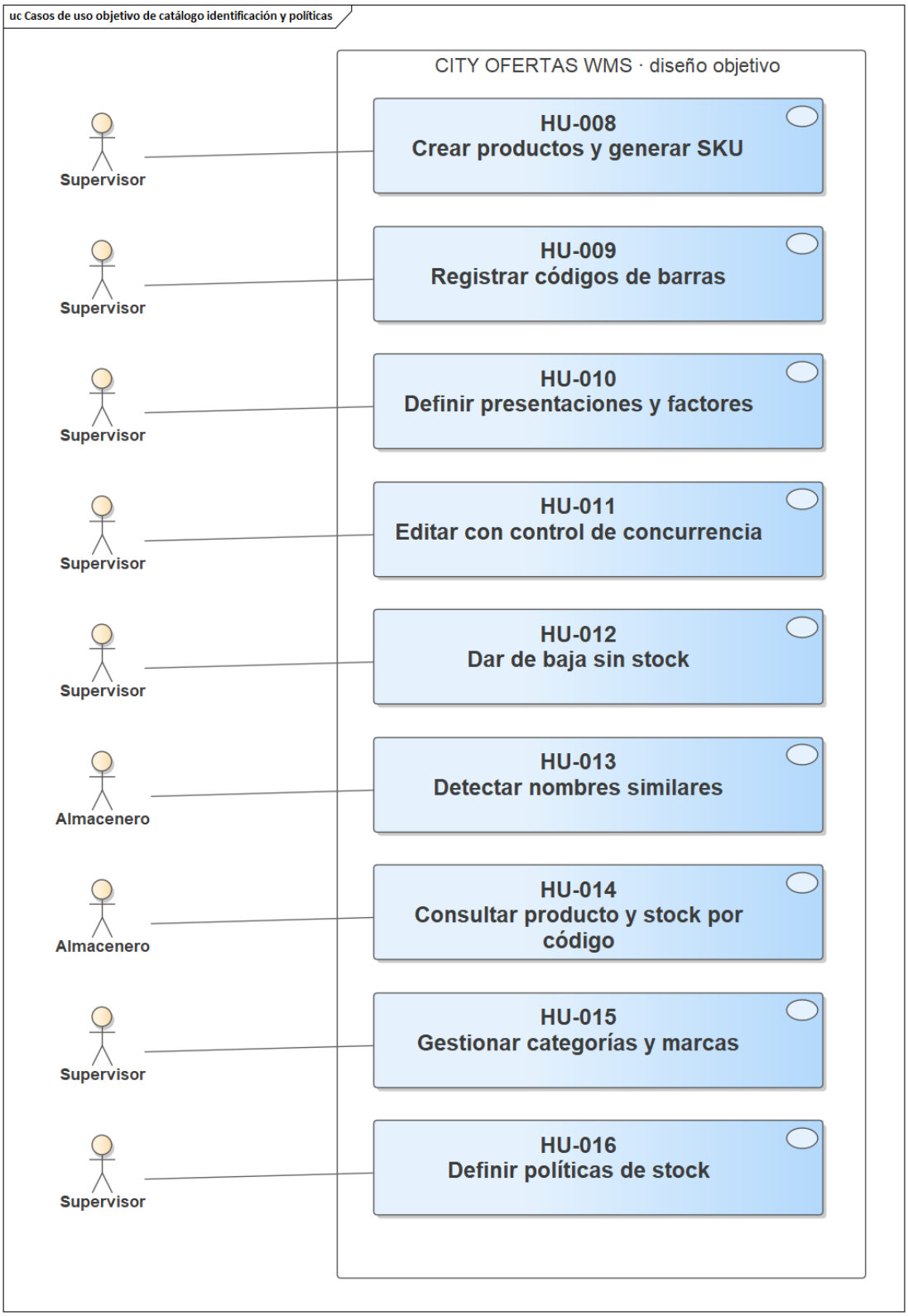
Las ocho vistas siguientes cubren las historias de todo el backlog, incluidas las futuras. Cada elemento conserva su ID y las notas del modelo editable remiten a criterios de aceptación. Se utiliza notación rectangular de casos de uso con su icono UML. Los actores se repiten por historia para mantener asociaciones legibles y representan el mismo rol funcional; su autorización definitiva requiere la validación de la matriz de permisos. Los escenarios alternativos y de error se detallan en el Excel, de modo que los diagramas no sustituyen la especificación de aceptación.

Figura 2 Casos de uso objetivo de acceso usuarios y auditoría



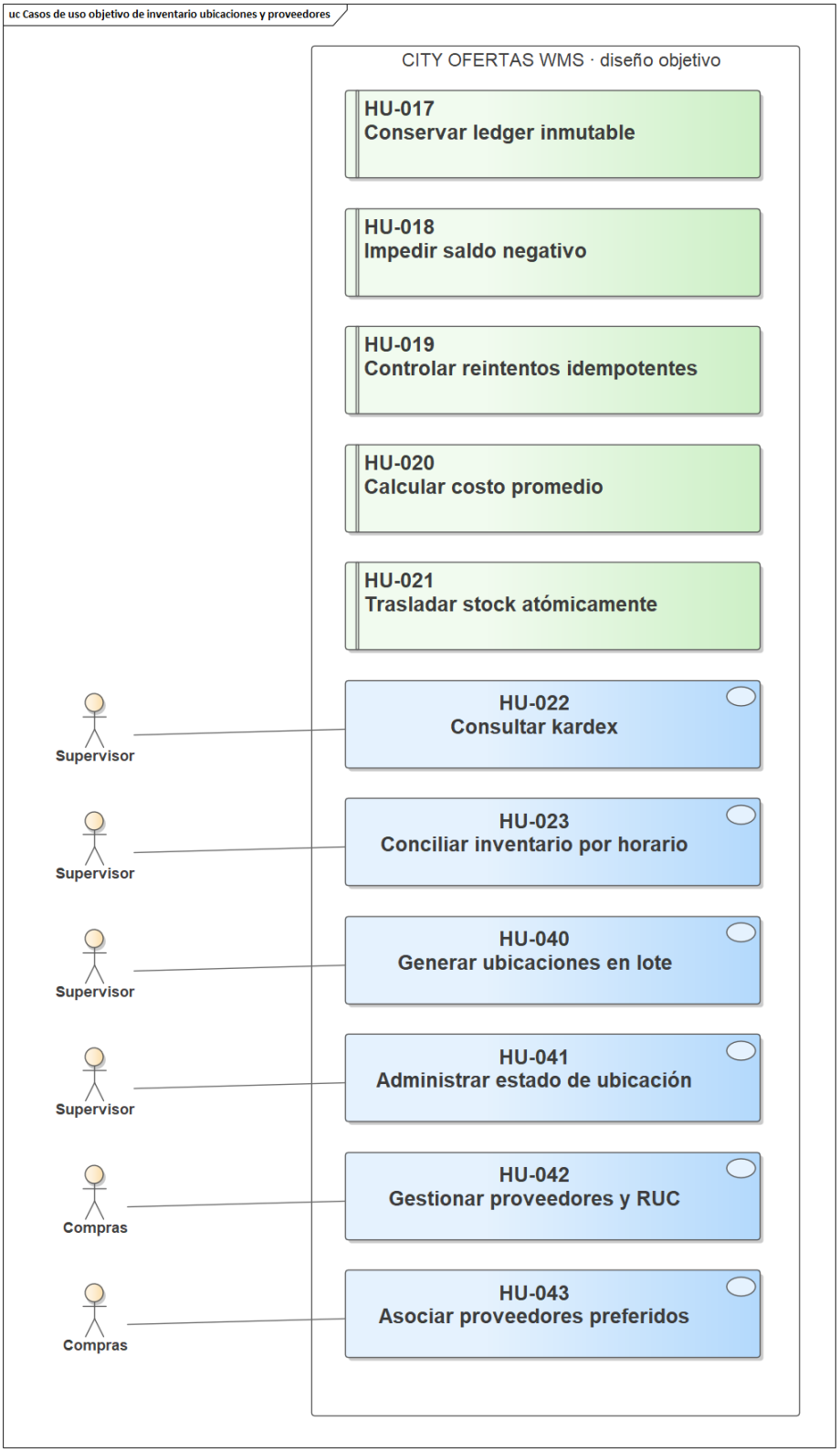
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 3 Casos de uso objetivo de catálogo identificación y políticas



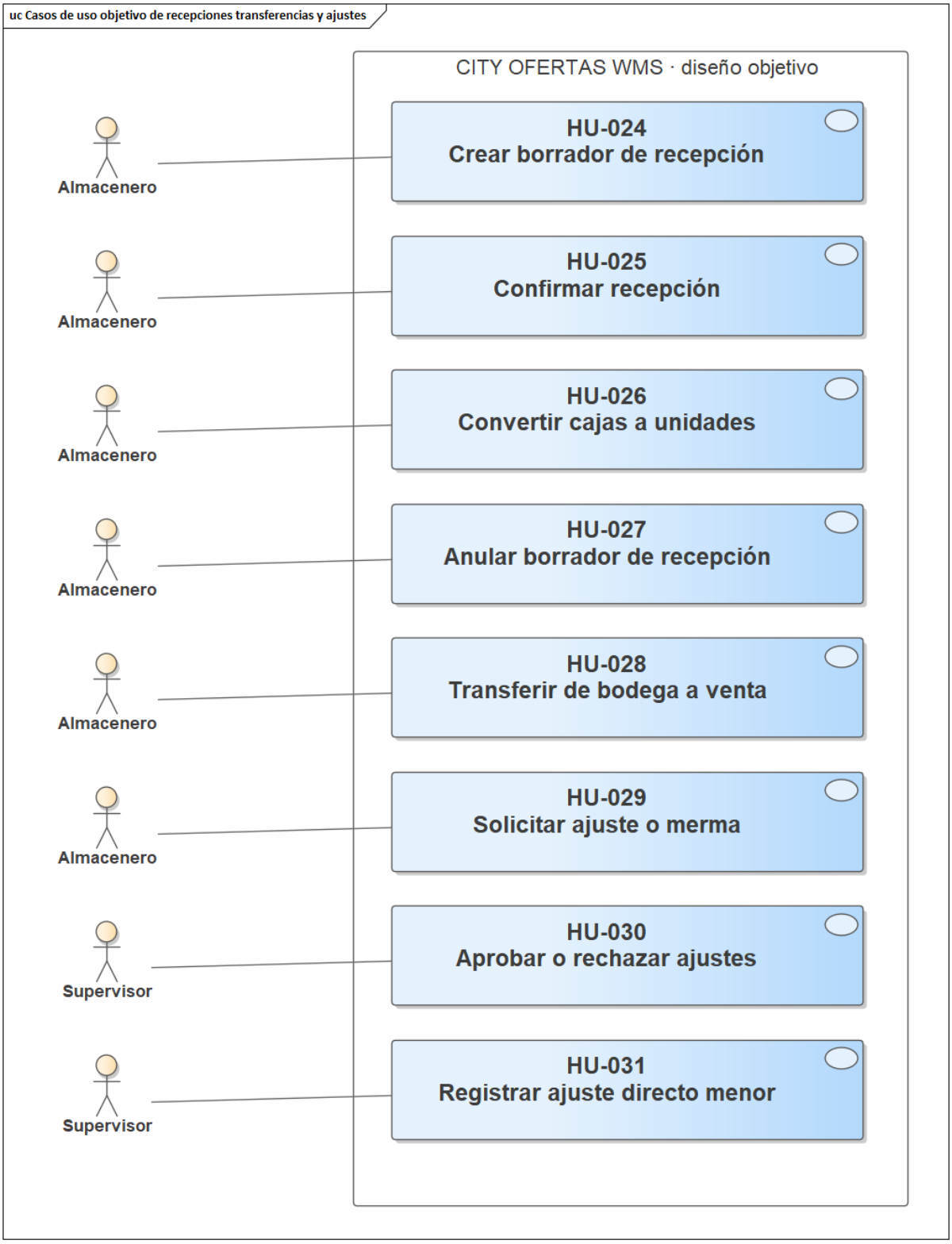
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 4 Casos de uso objetivo de inventario ubicaciones y proveedores



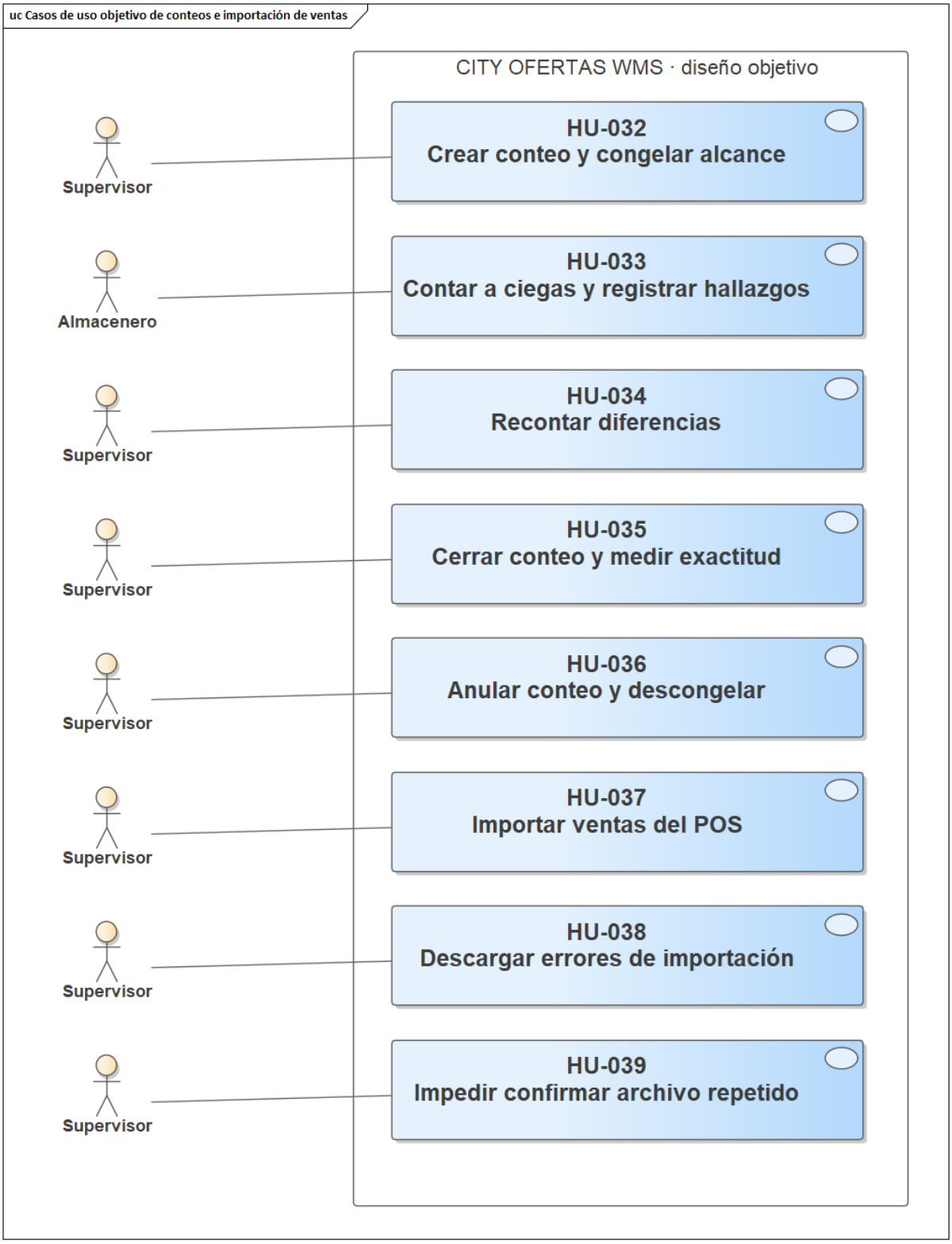
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 5 Casos de uso objetivo de recepciones transferencias y ajustes



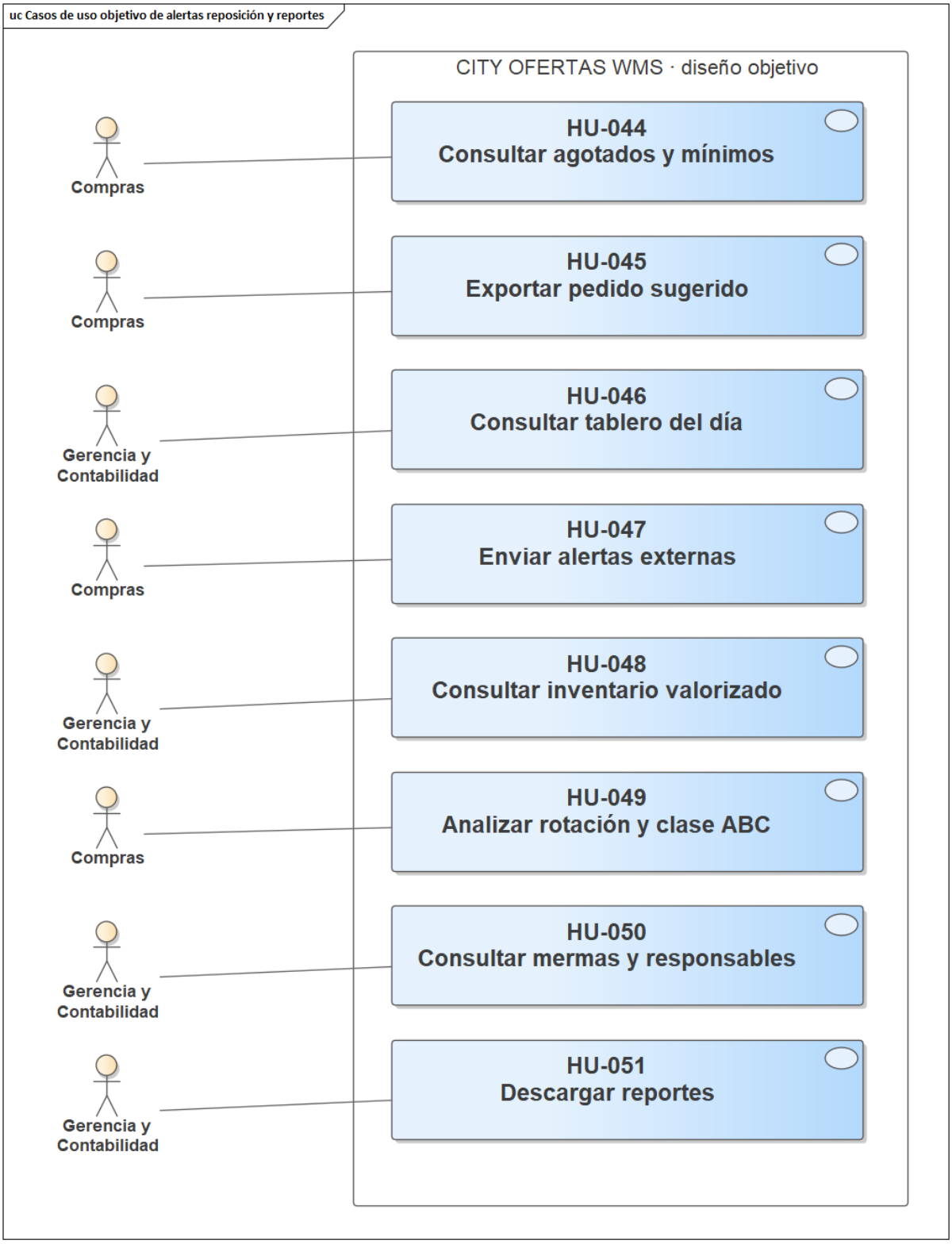
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 6 Casos de uso objetivo de conteos e importación de ventas



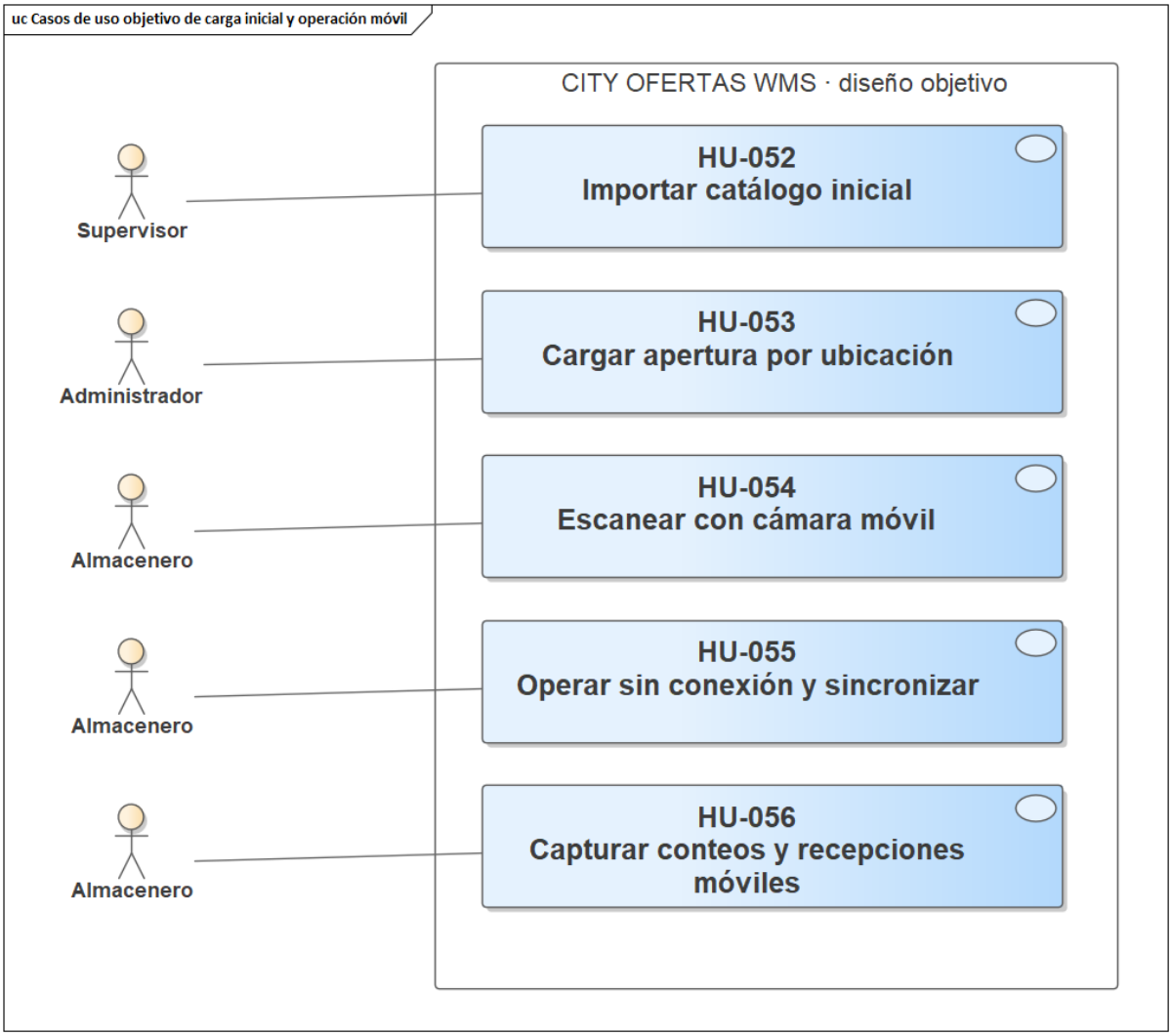
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 7 Casos de uso objetivo de alertas reposición y reportes



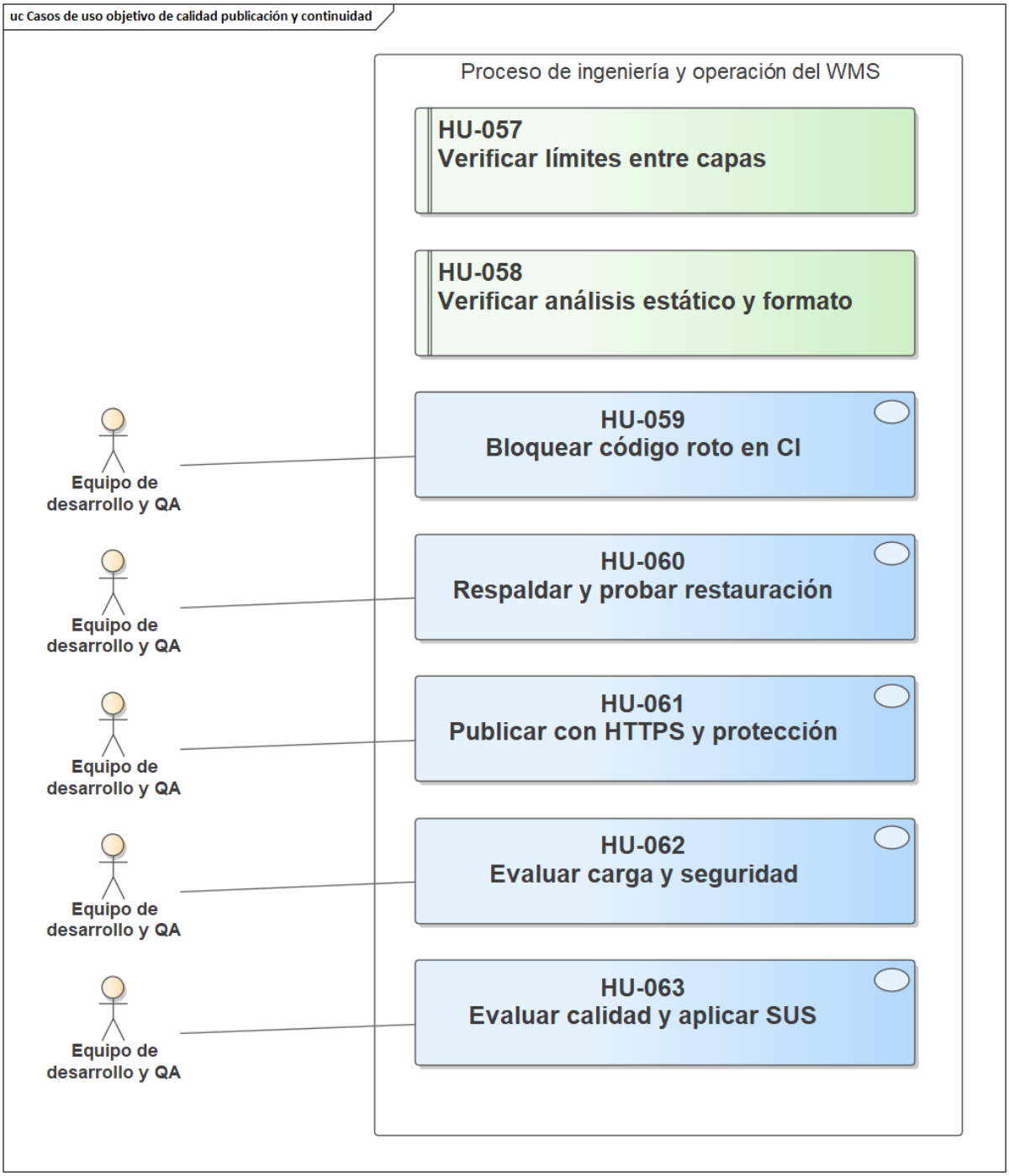
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 8 Casos de uso objetivo de carga inicial y operación móvil



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 9 Casos de uso objetivo de calidad publicación y continuidad



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

5.2 Arquitectura del sistema

5.2.1 Descripción

La implementación corresponde a un monolito modular Laravel. Los siete módulos principales son Iam, Catalog, Inventory, Operations, Locations, Suppliers y Audit. En el conjunto revisado se identificaron 169 clases, interfaces y enumeraciones distribuidas entre Domain, Application e Infrastructure. El panel Filament y Livewire reside en app/Filament; los modelos y políticas de aplicación complementan los módulos.

La capa de dominio contiene valores, reglas y contratos. La aplicación coordina casos de uso y transacciones. La infraestructura implementa persistencia, eventos y adaptadores. El panel invoca comportamientos y consulta modelos o vistas de PostgreSQL. Algunas lecturas de Filament acceden directamente a vistas de datos; por ello, el diseño se describe como modular y con puertos en los casos relevantes, sin afirmar una separación absoluta de todas las consultas de presentación.

5.2.2 Justificación

El monolito modular permite desplegar una sola aplicación y mantener límites de responsabilidad sin introducir la complejidad operativa de servicios distribuidos. Los puertos facilitan probar las reglas con dobles en memoria y verificar por separado la implementación PostgreSQL. El ledger y los saldos se actualizan dentro de transacciones, con bloqueos cuando corresponde, para preservar consistencia durante recepciones, salidas, traslados y conteos.

Las funciones futuras se incorporan como contratos de diseño. Se proponen importadores separados de catálogo y apertura, una tarea programada de conciliación, un despachador de alertas y procesos de copia y restauración. El cliente móvil objetivo usa una cola local de pendientes y un servicio de sincronización con validación de conflictos. IndexedDB y esos contratos móviles son decisiones propuestas; no son tablas PostgreSQL ya implementadas ni capacidades verificadas del panel actual.

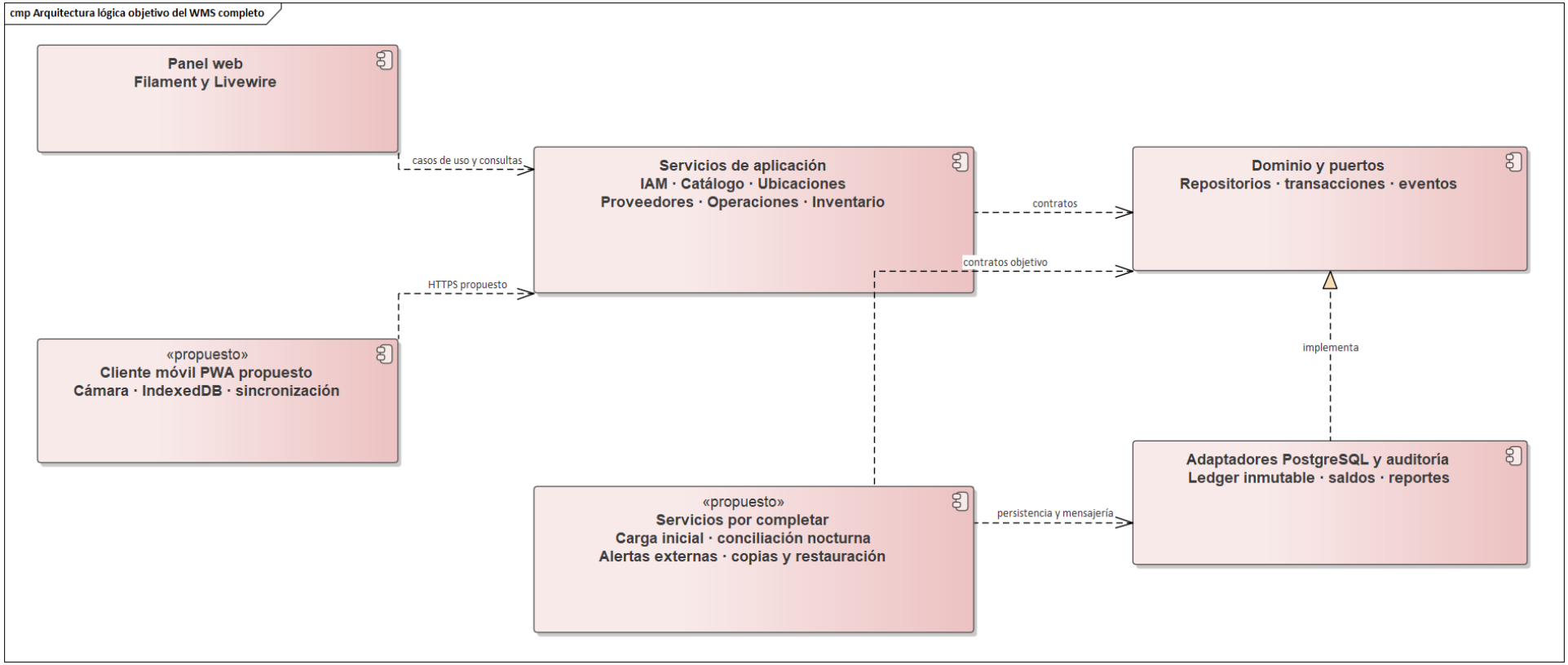
5.2.3 Diagramas UML

5.2.3.1 Arquitectura lógica

5.2.3.1.1 Diagrama de componentes

La vista de componentes relaciona el panel, los servicios de aplicación, el dominio y los adaptadores. El cliente móvil y los servicios pendientes aparecen identificados como propuestos. La dependencia hacia contratos no implica que ya exista un endpoint público para todas las integraciones objetivo.

Figura 10 Arquitectura lógica objetivo del WMS completo



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

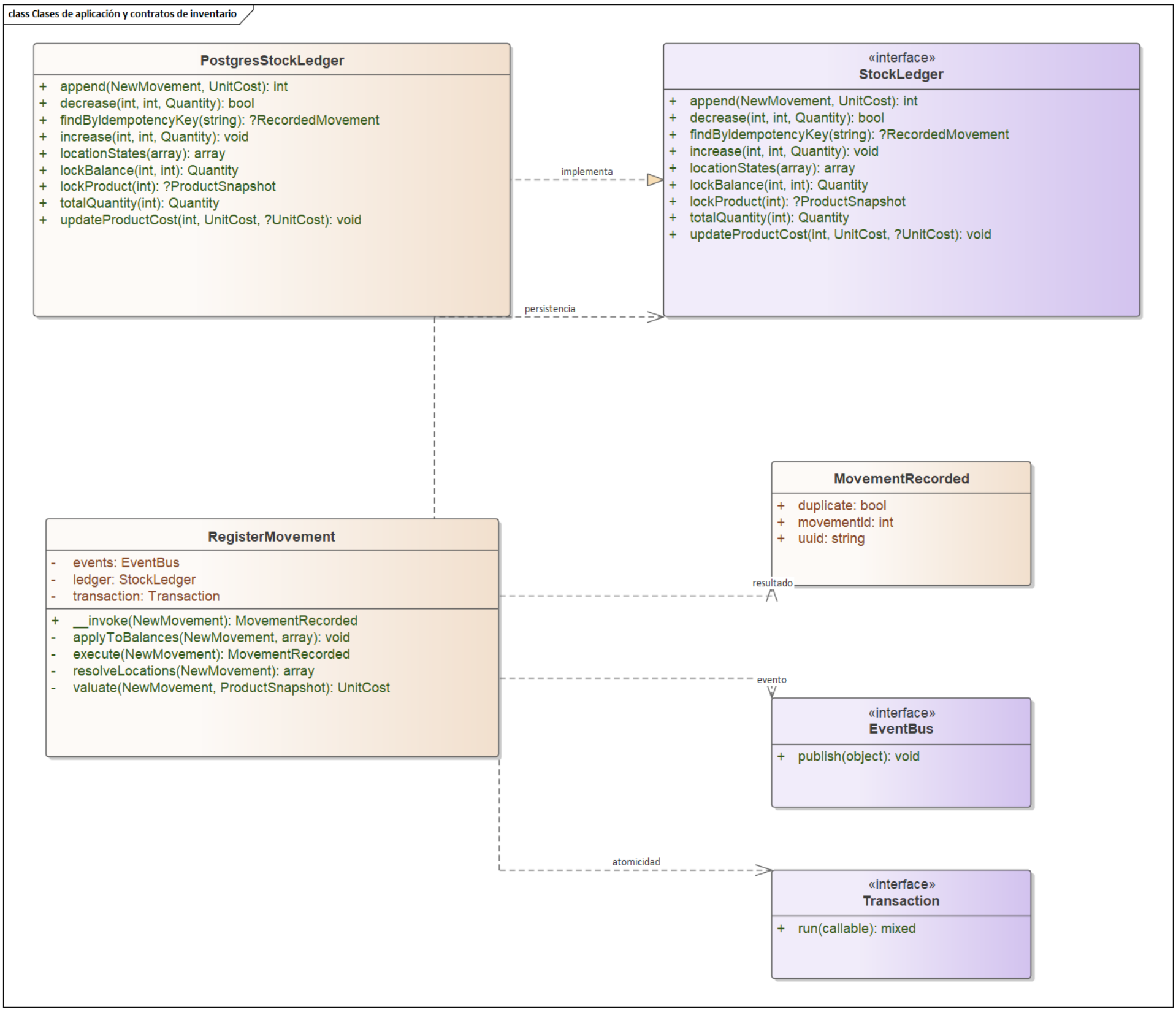
5.2.3.1.2 Diagramas de clases

Las vistas de clases muestran atributos, visibilidad, operaciones, parámetros, tipos y relaciones relevantes. Las láminas de clases y datos utilizan formato A3 para conservar la legibilidad del detalle; el texto principal mantiene A4. El modelo editable incorpora las 169 definiciones extraídas del código y 13 contratos propuestos. Las firmas de constructores extensas se conservan en las notas de los elementos para evitar saturar las figuras. Estas clases propuestas especifican el diseño pendiente y no se incluyen en los resultados de cobertura del código existente.

RegisterMovement coordina la transacción, consulta el ledger, aplica los cambios de saldo y publica el evento. StockLedger define el contrato de persistencia y PostgresStockLedger su adaptación. NewMovement y los valores Quantity y UnitCost encapsulan los datos y precisión del movimiento. Los diagramas de recepción y de movilidad completan los procesos de entrada y la evolución prevista.

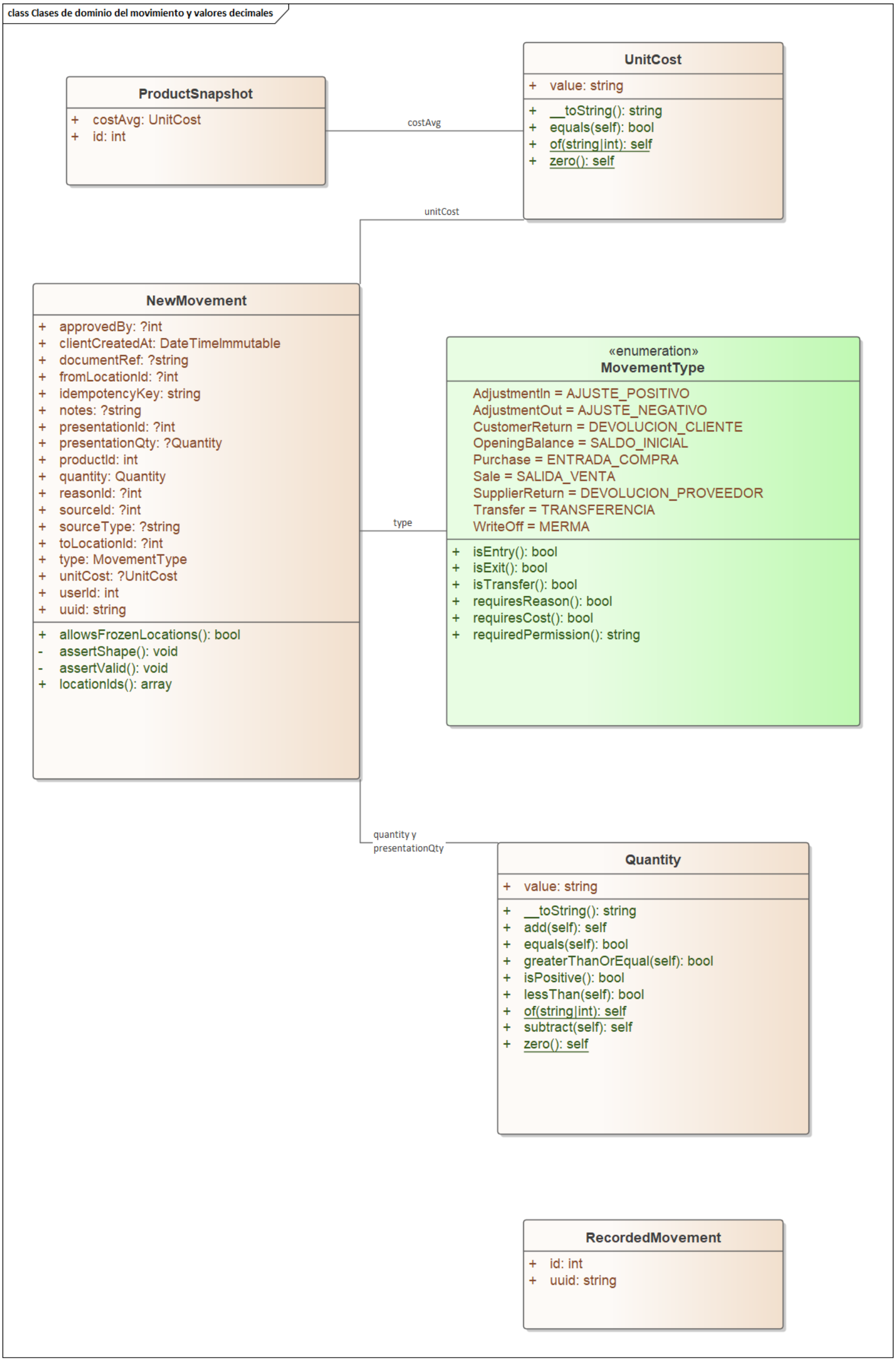
Las vistas complementarias detallan usuarios y autenticación, catálogo, ubicaciones, proveedores, auditoría, conteos, POS, transferencias, ajustes y continuidad. Sus relaciones proceden de atributos, firmas y adaptadores de las clases del proyecto. Las dos clases de continuidad son propuestas y se identifican como tales.

Figura 11 Clases de aplicación y contratos de inventario



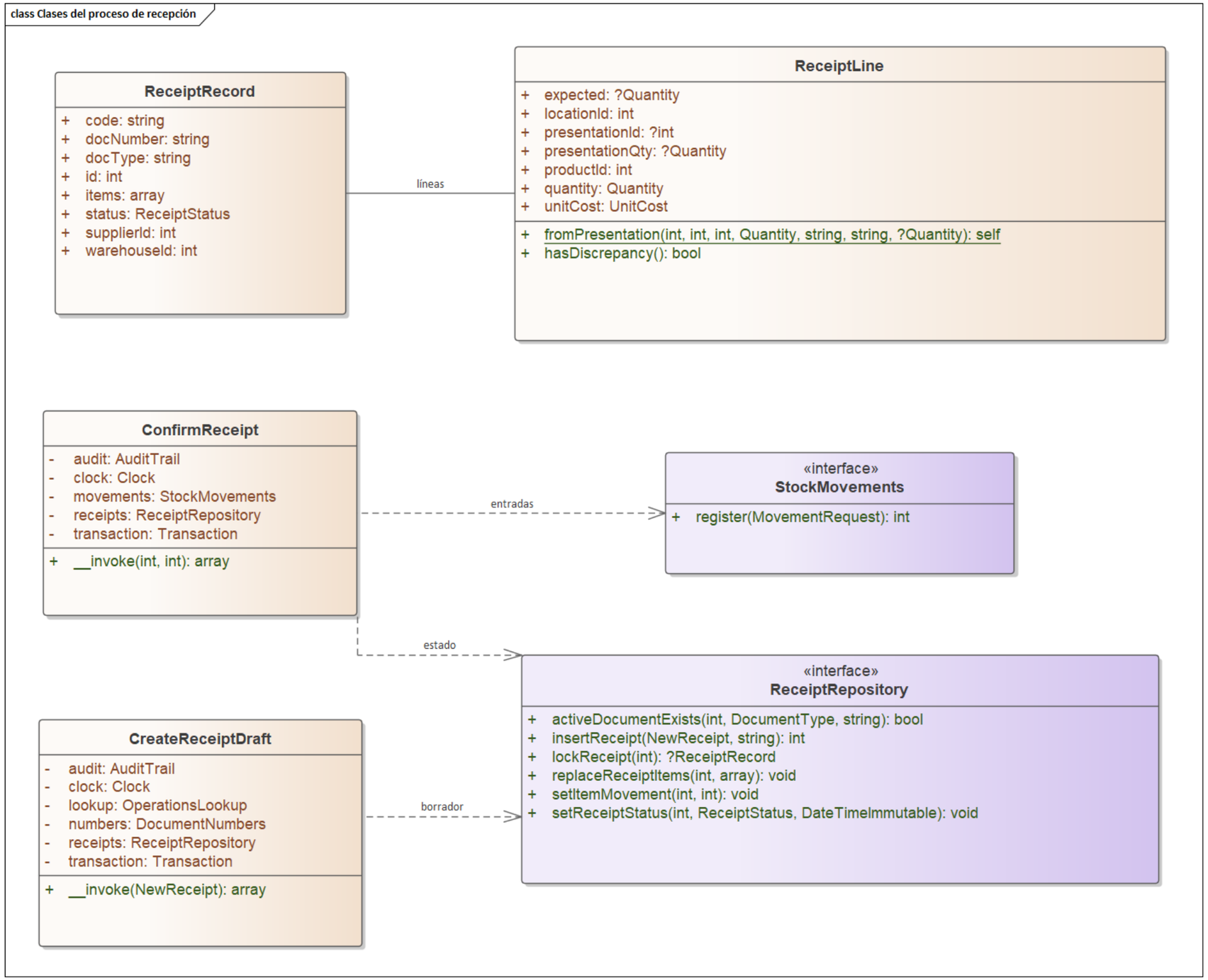
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 12 Clases de dominio del movimiento y valores decimales



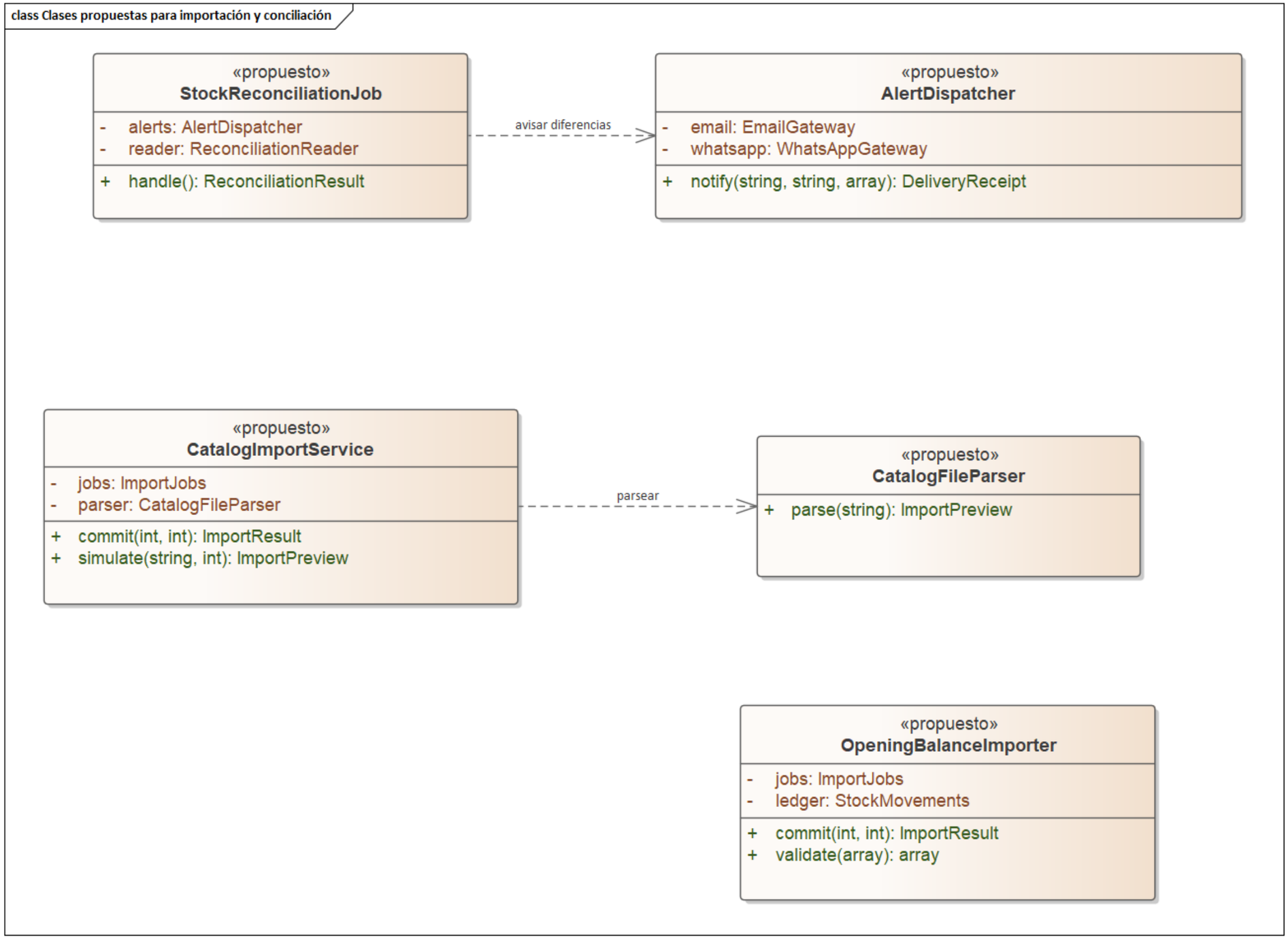
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 13 Clases del proceso de recepción



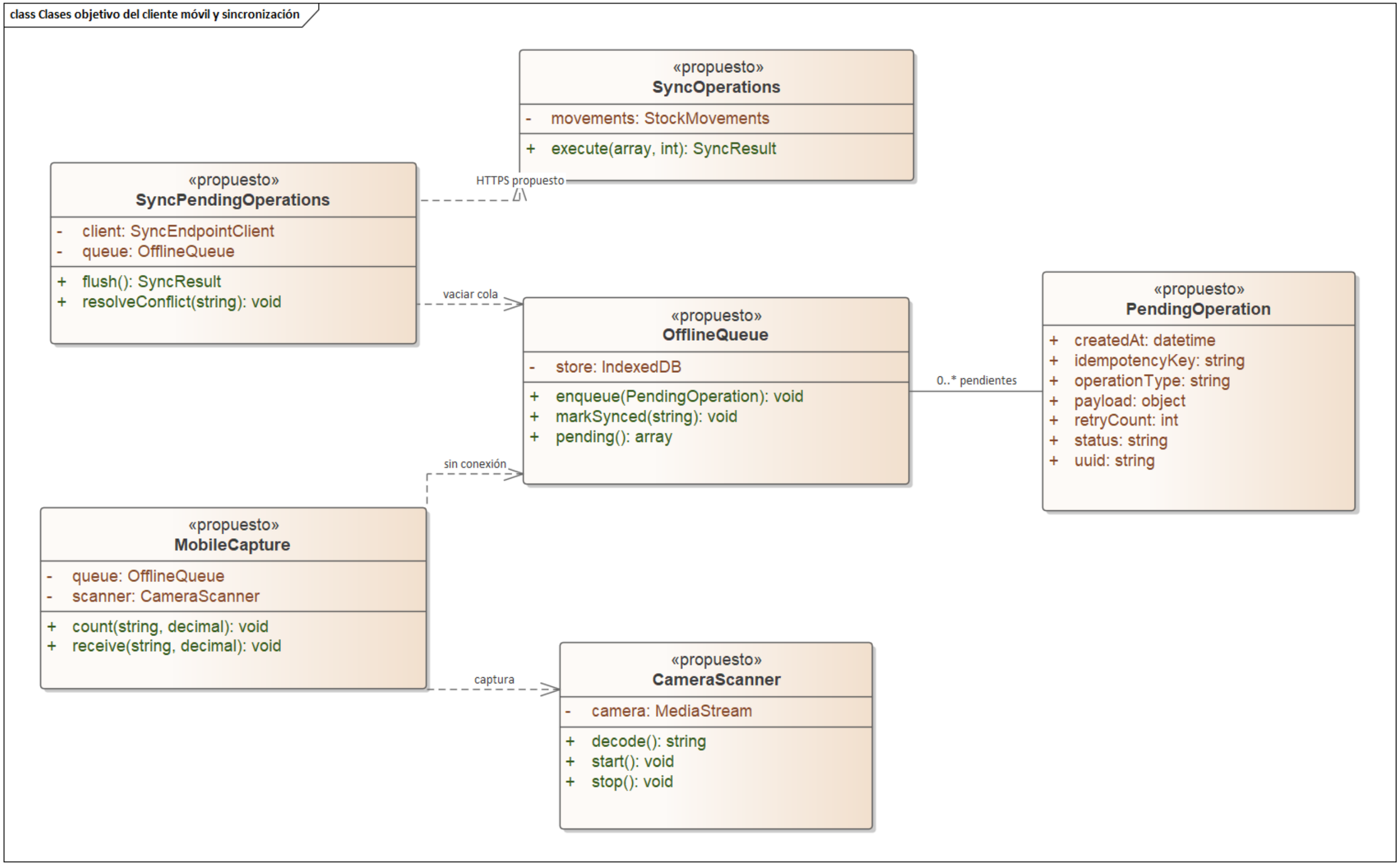
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 14 Clases propuestas para importación y conciliación



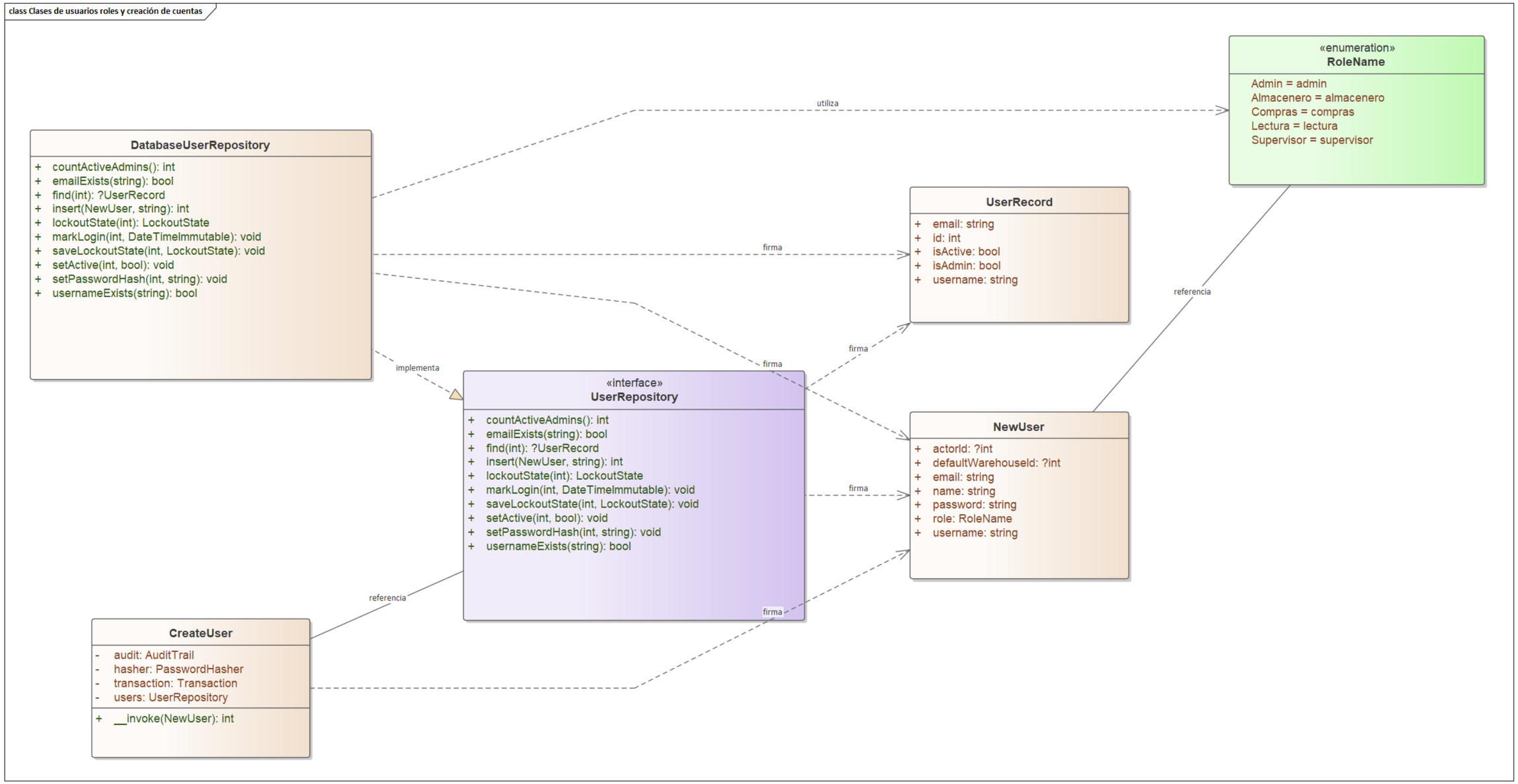
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 15 Clases objetivo del cliente móvil y sincronización



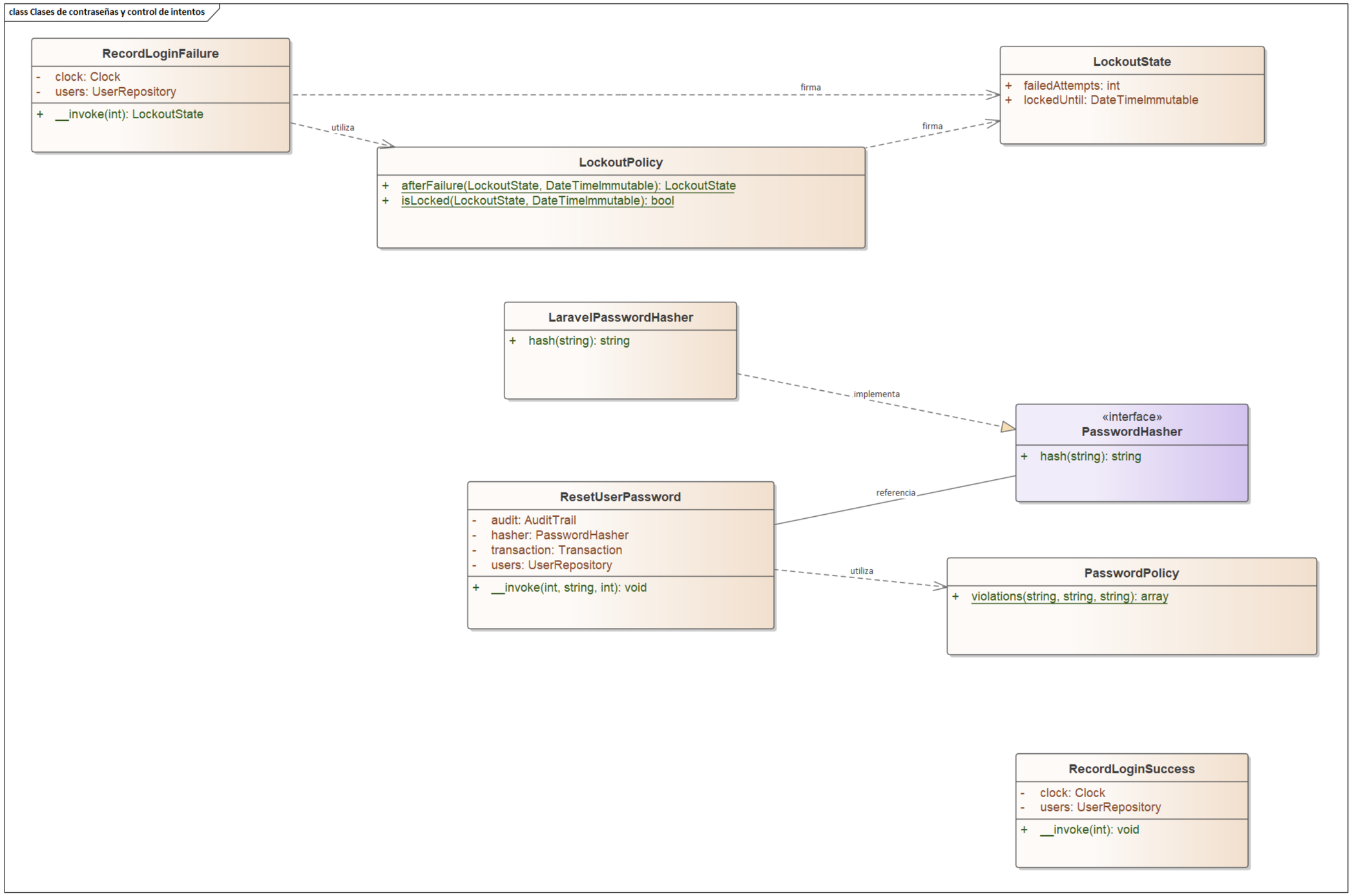
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 16 Clases de usuarios roles y creación de cuentas



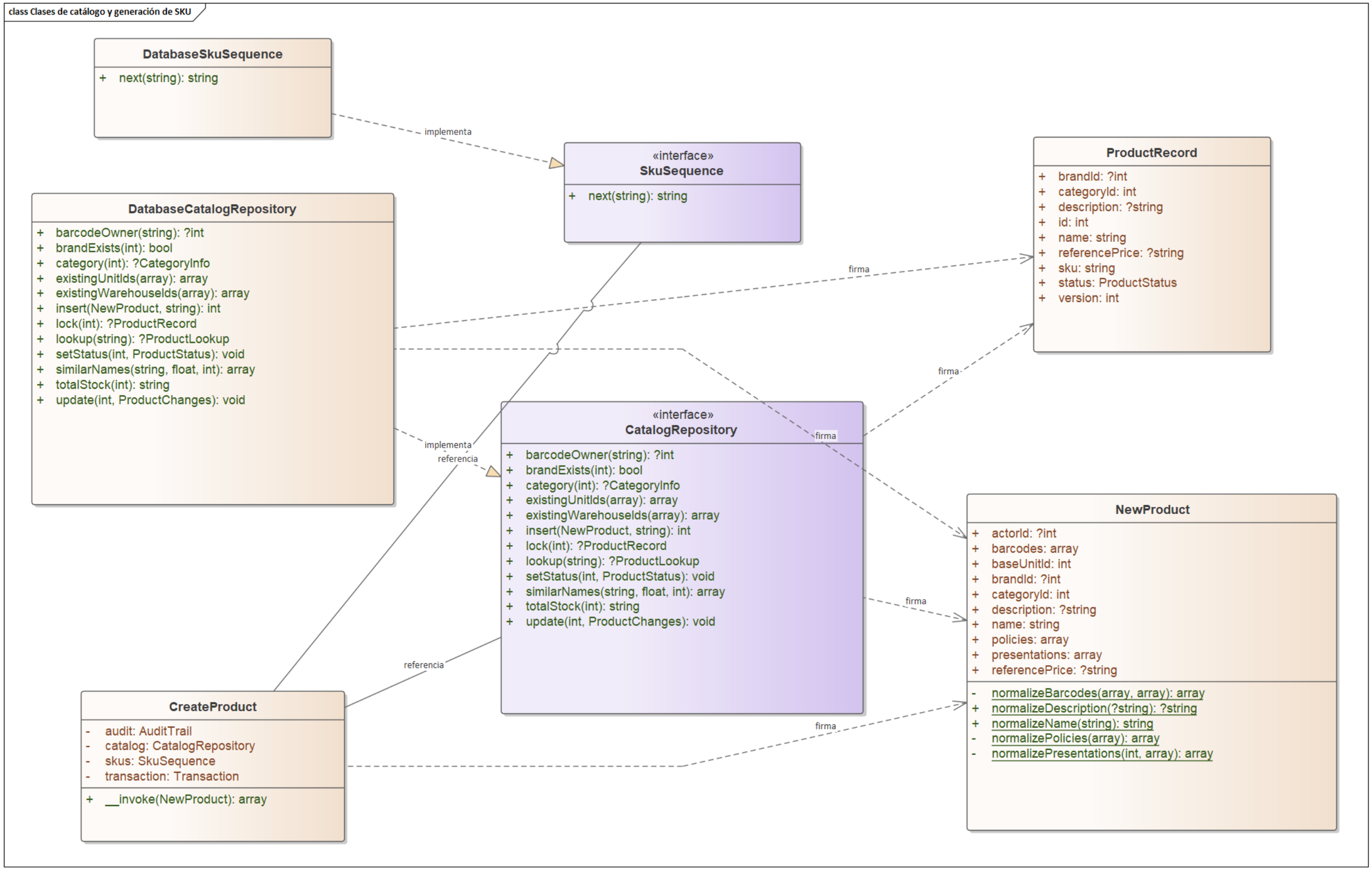
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 17 Clases de contraseñas y control de intentos



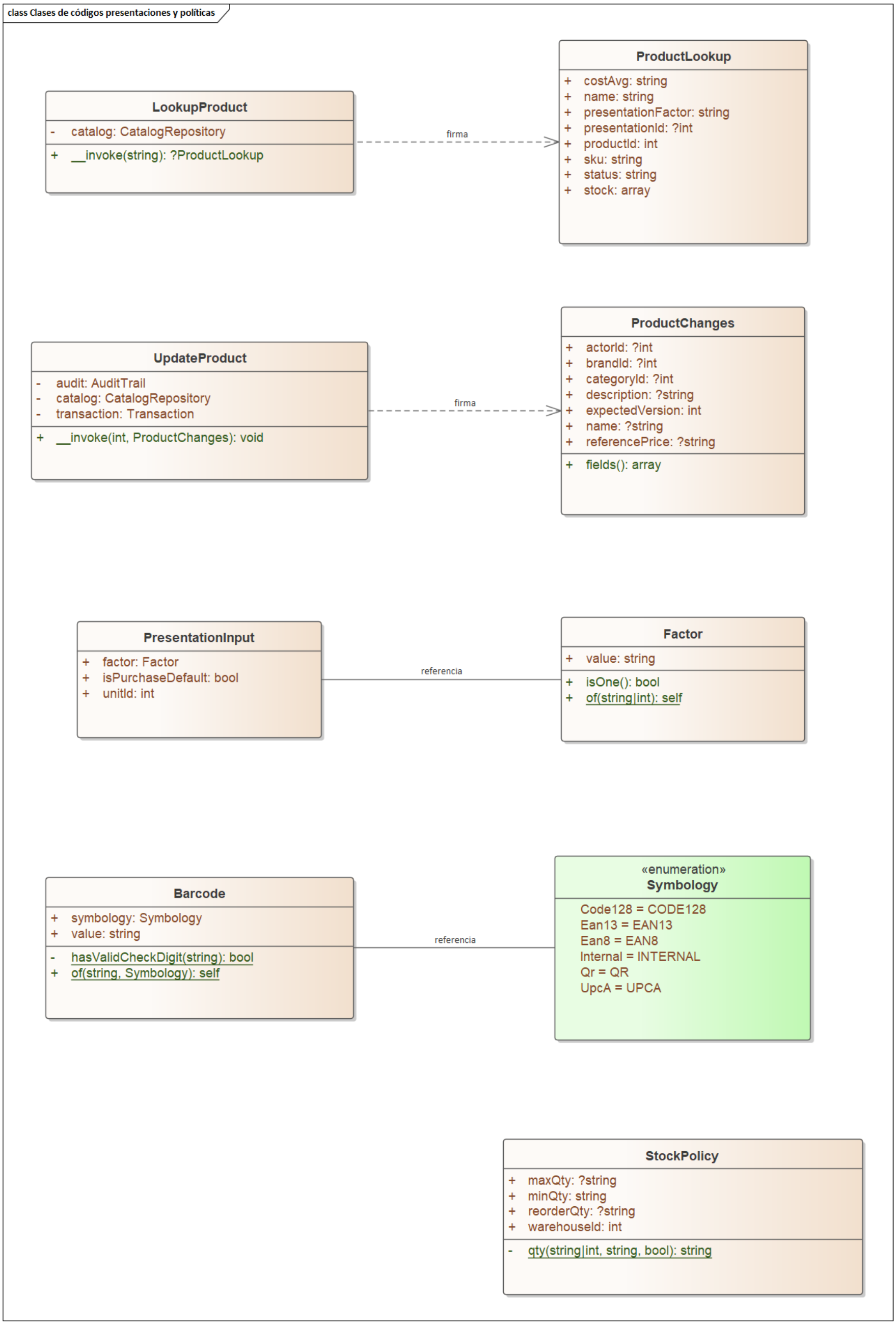
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 18 Clases de catálogo y generación de SKU



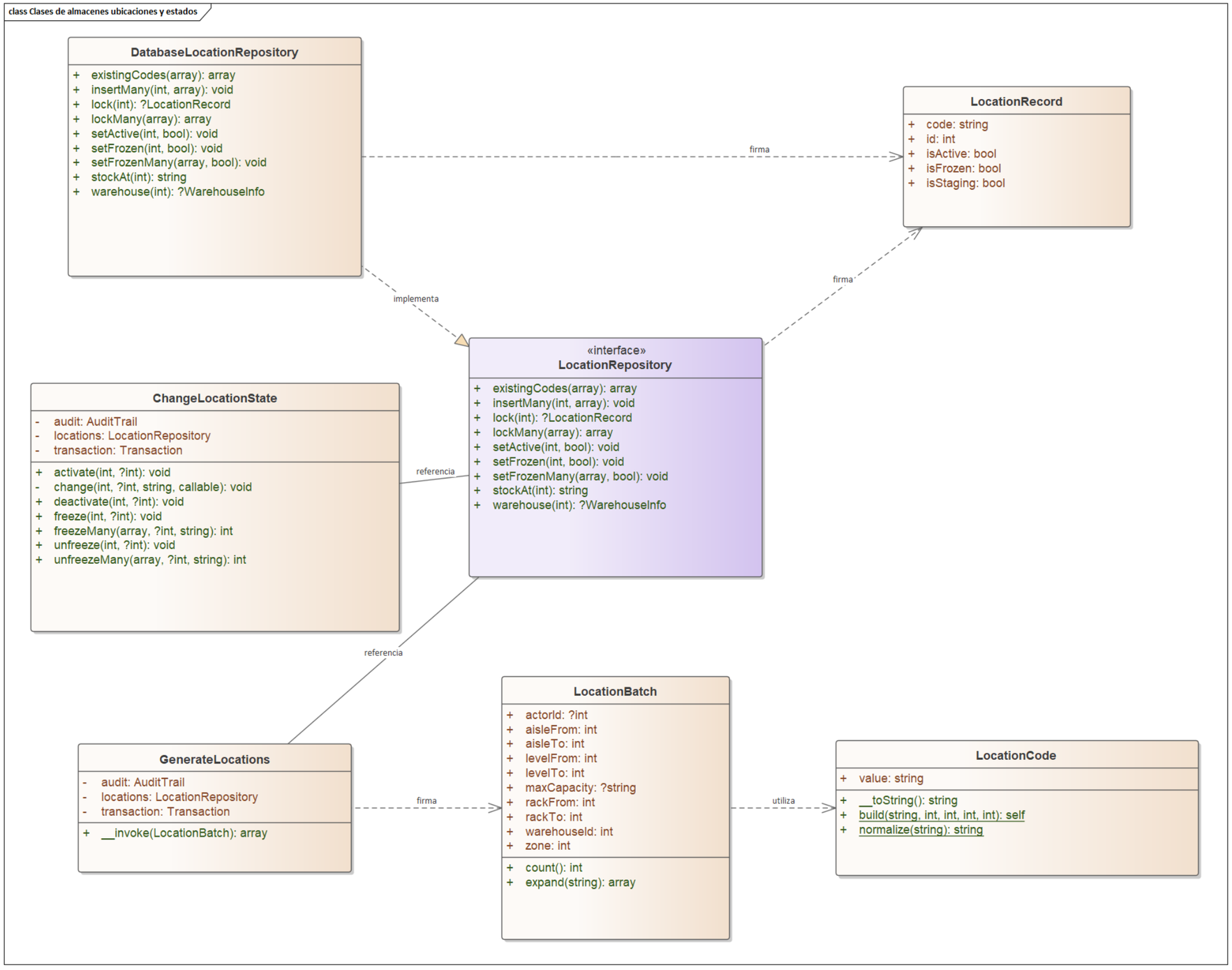
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 19 Clases de códigos presentaciones y políticas



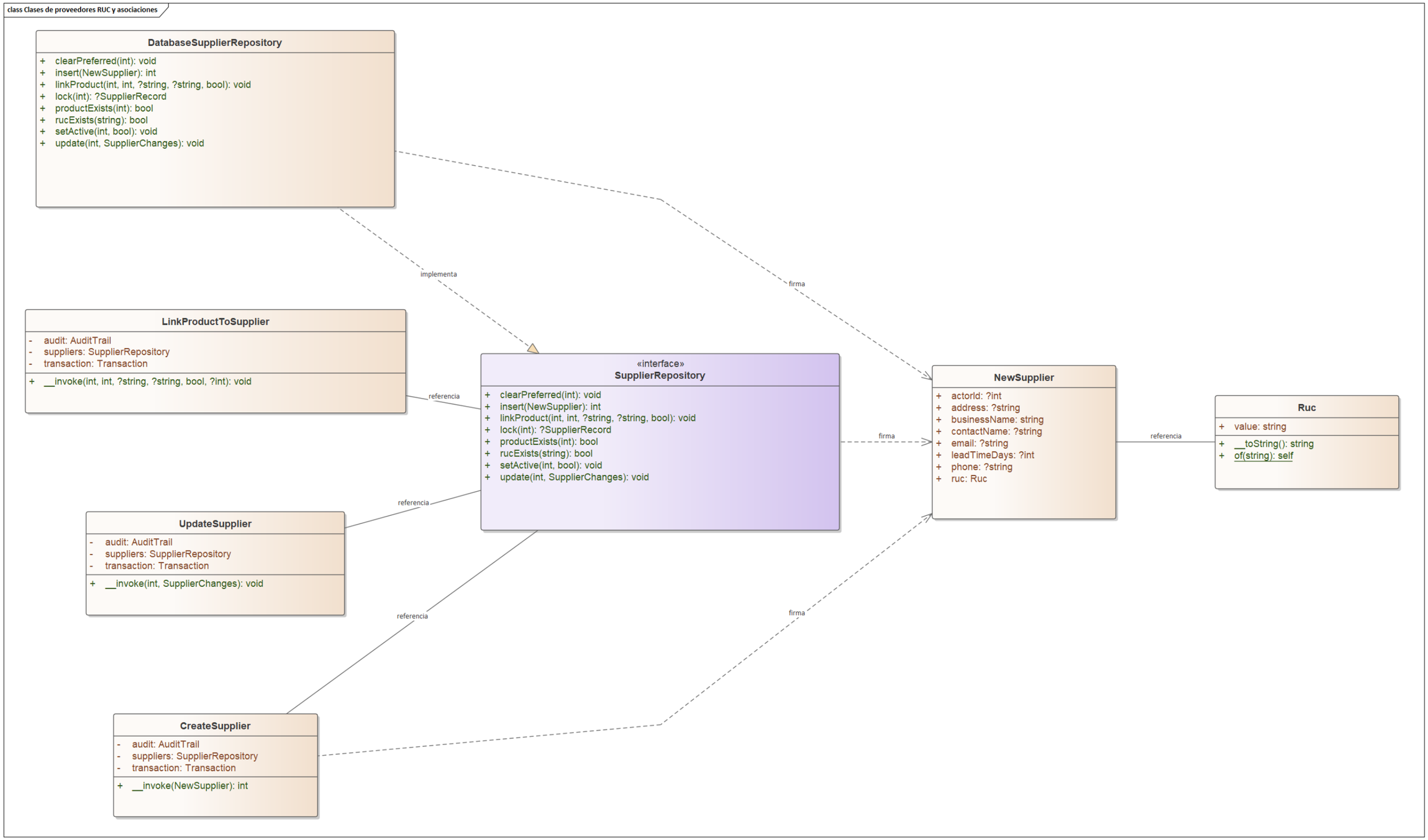
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 20 Clases de almacenes ubicaciones y estados



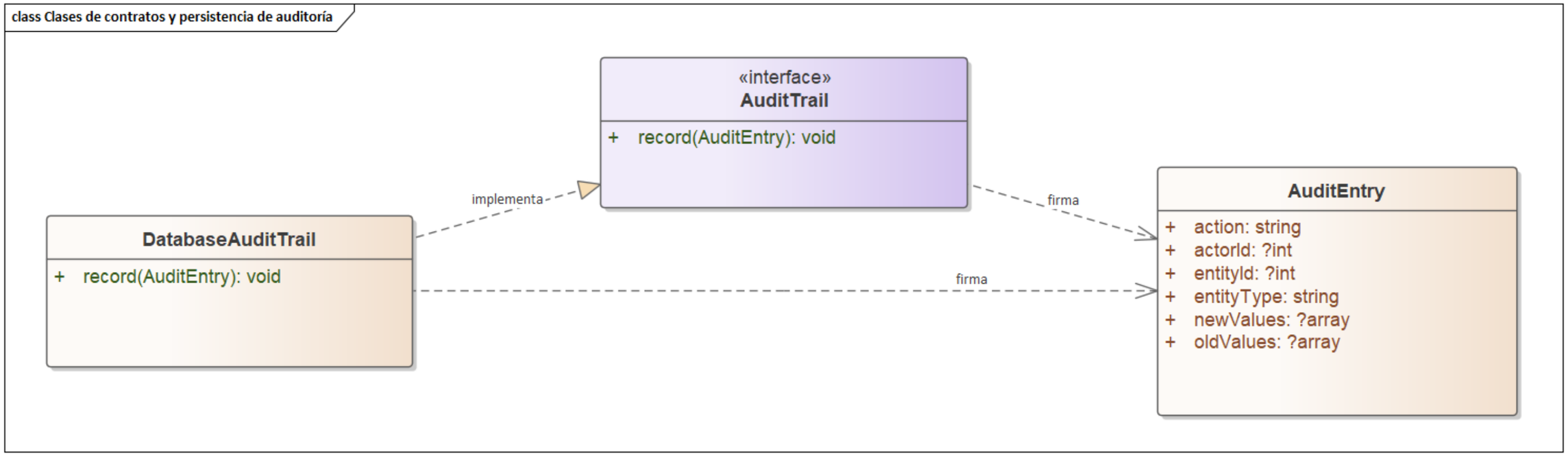
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 21 Clases de proveedores RUC y asociaciones



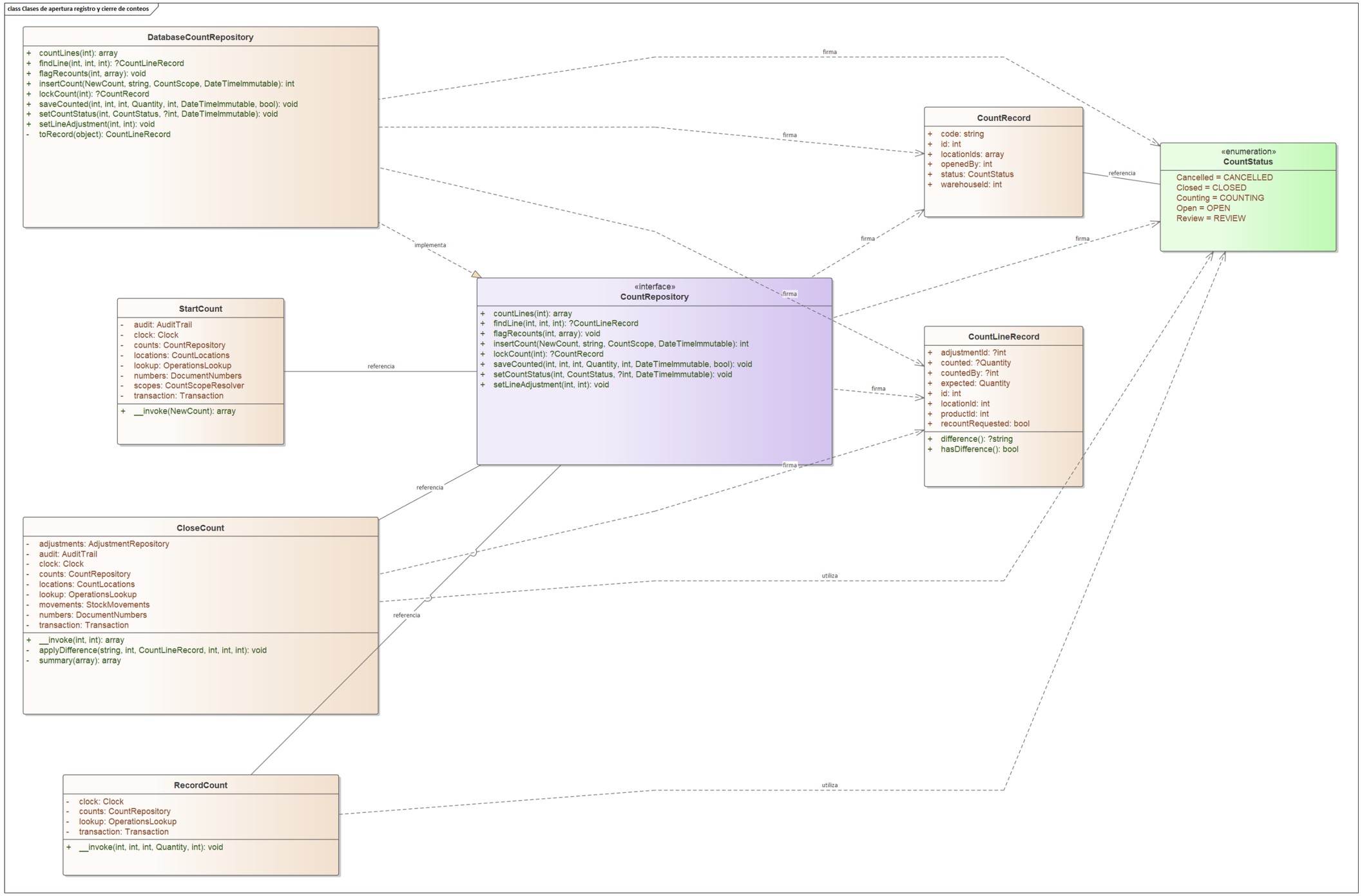
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 22 Clases de contratos y persistencia de auditoría



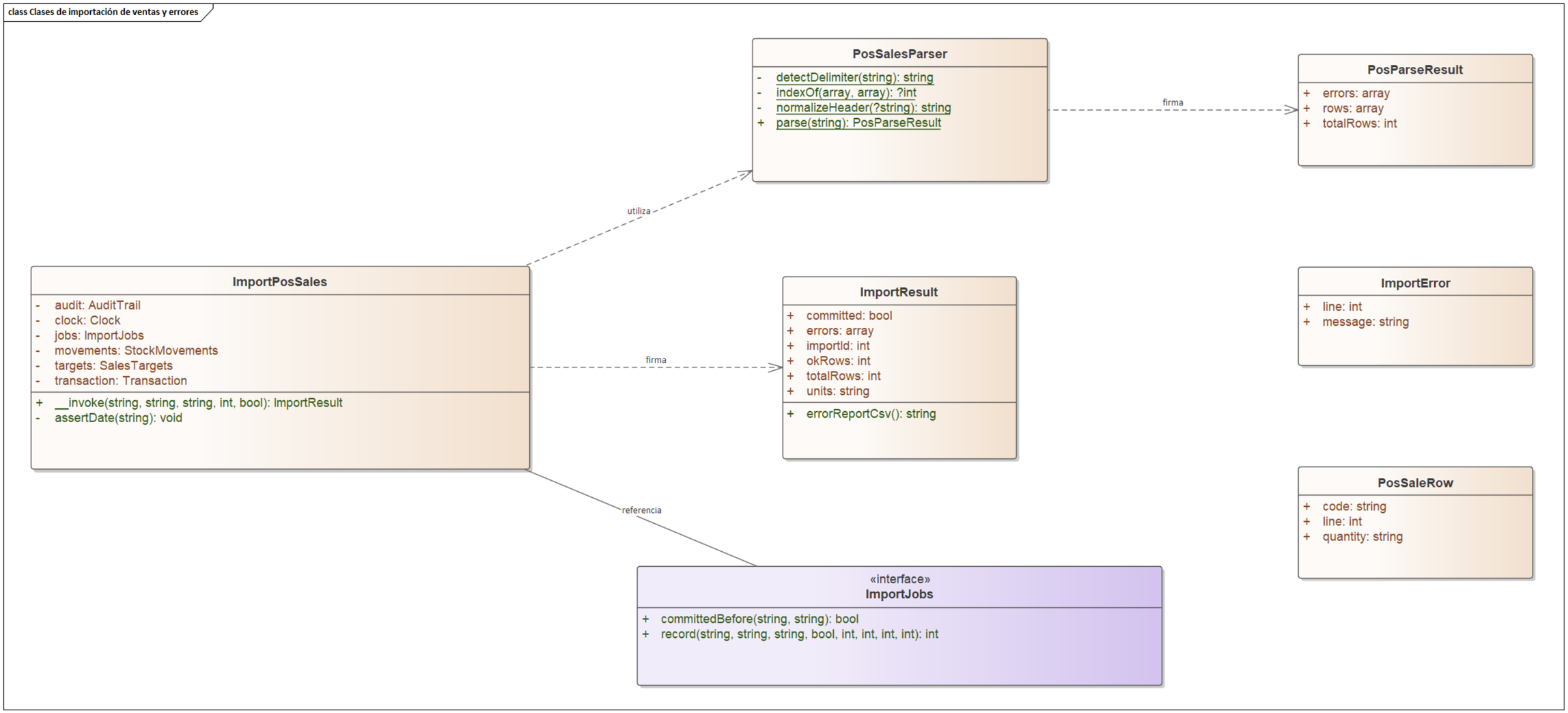
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 23 Clases de apertura registro y cierre de conteos



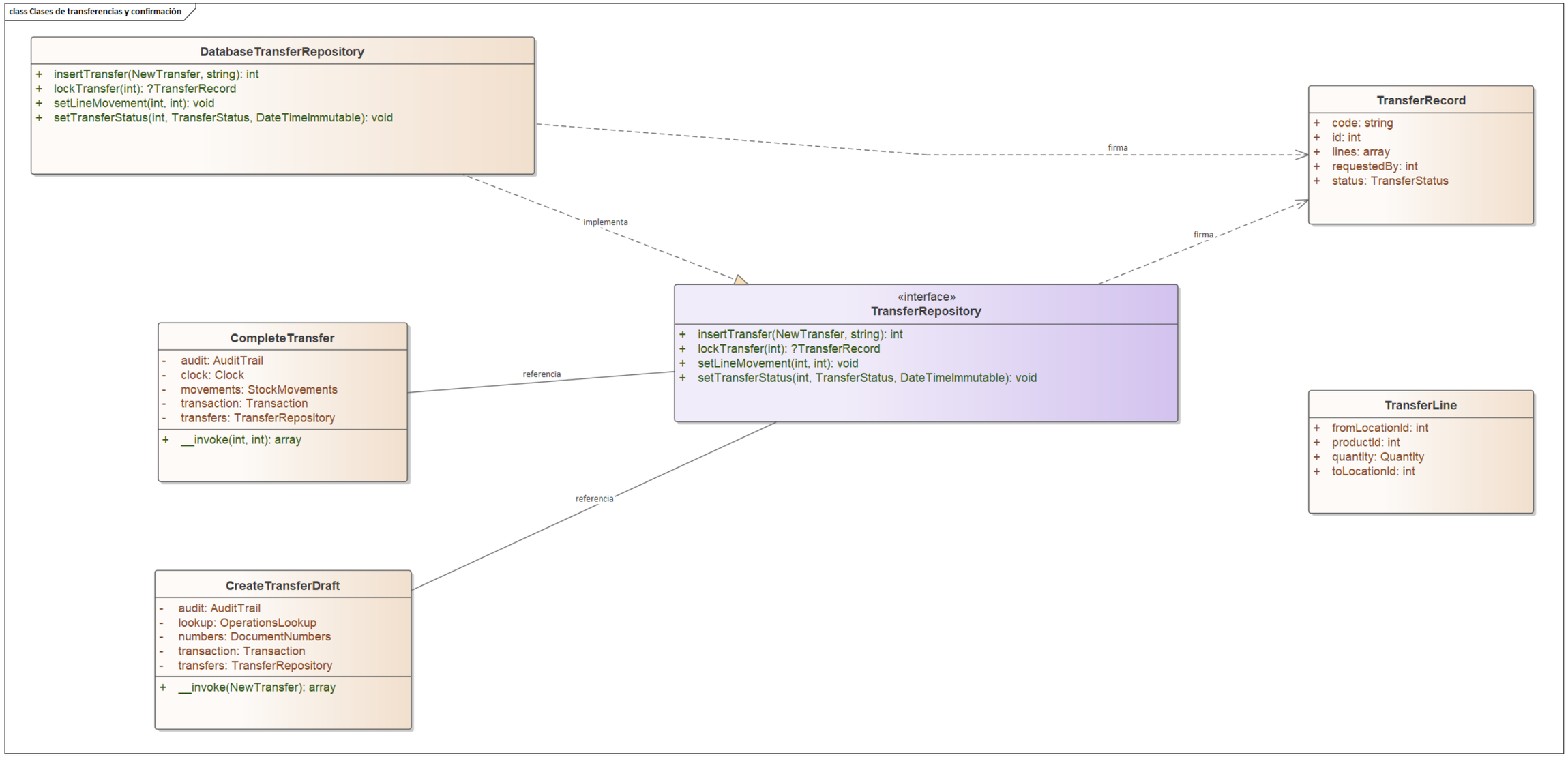
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 24 Clases de importación de ventas y errores



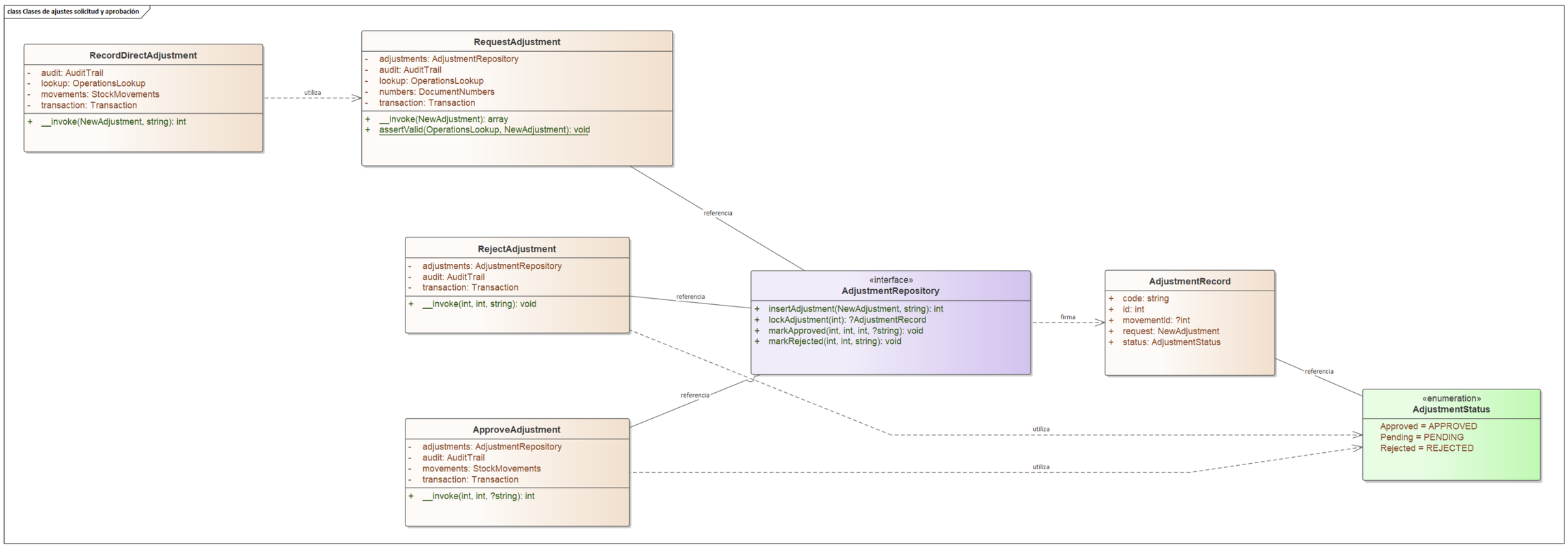
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 25 Clases de transferencias y confirmación



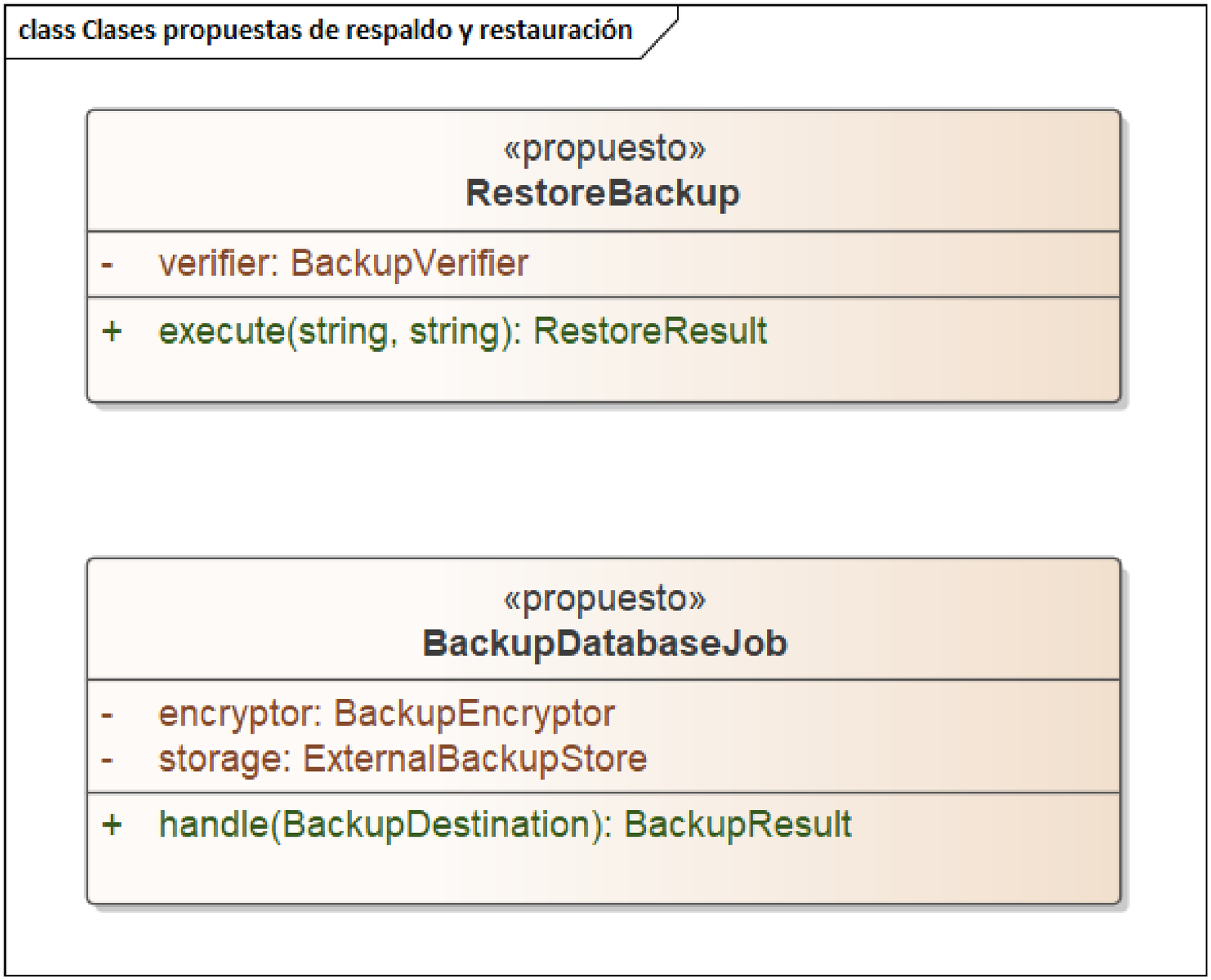
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 26 Clases de ajustes solicitud y aprobación



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 27 Clases propuestas de respaldo y restauración



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

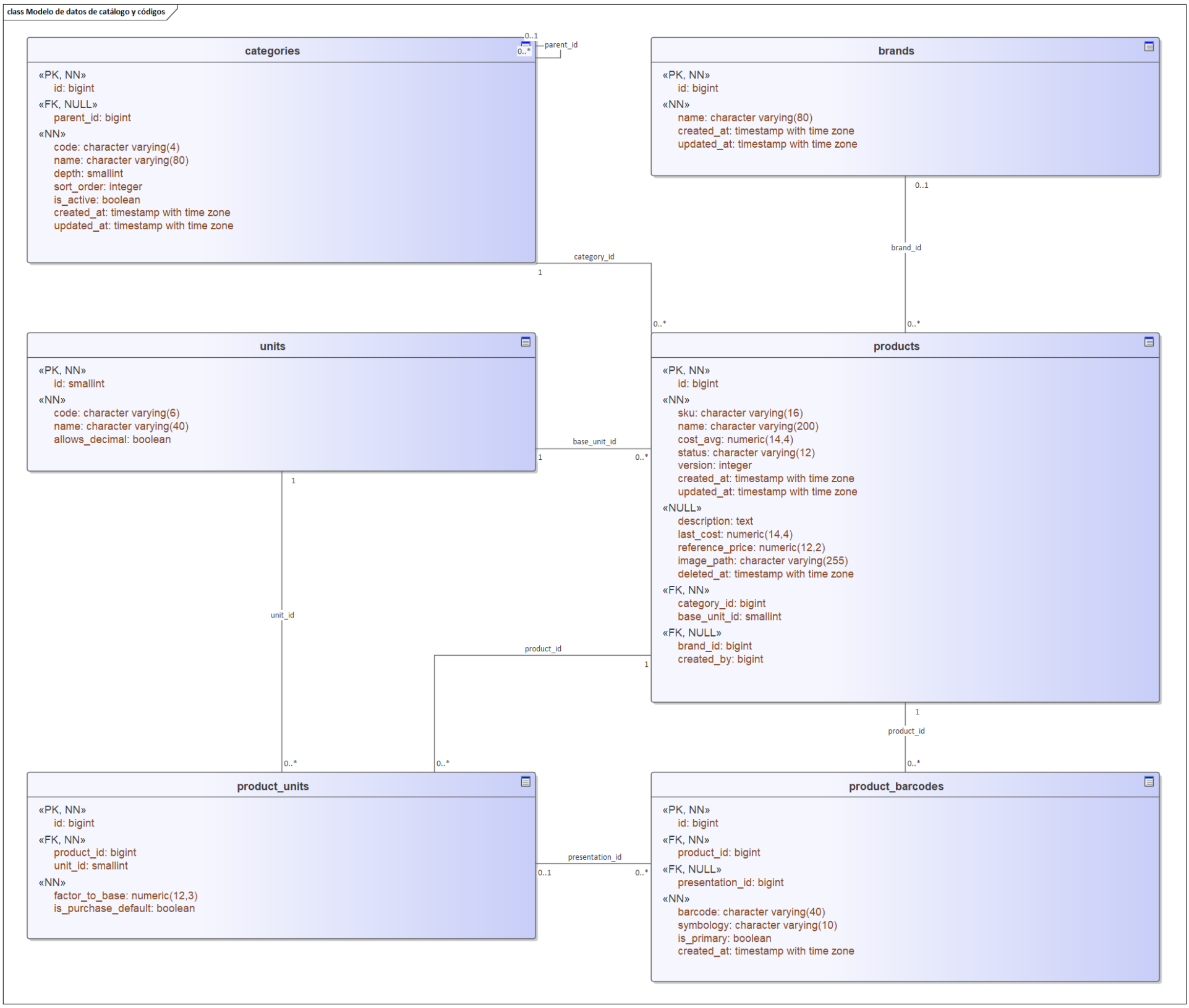
5.2.3.1.3 Modelo de datos y reglas de integridad

La extracción estructural de la base revisada identificó 46 tablas físicas, incluidas infraestructura y una partición, con 343 columnas y 66 claves foráneas. No se cuentan las vistas como tablas de negocio. El modelo conserva los tipos efectivos con longitud o precisión, las claves primarias, las referencias y la nulabilidad. NN significa no nulo y NULL indica que el campo admite ausencia de valor.

El catálogo se relaciona con categorías, marcas, unidades, presentaciones y códigos. El inventario relaciona productos y ubicaciones con movimientos y saldos. Las operaciones disponen de cabeceras y detalles, y los conteos incluyen líneas y el conjunto de ubicaciones afectadas. IAM utiliza usuarios, roles, permisos y tablas de asociación; auditoría, importaciones y alertas conservan registros de soporte. Las figuras se dividen por área para permitir leer los atributos; las referencias que atraviesan áreas se consultan en el modelo y diccionario adjuntos.

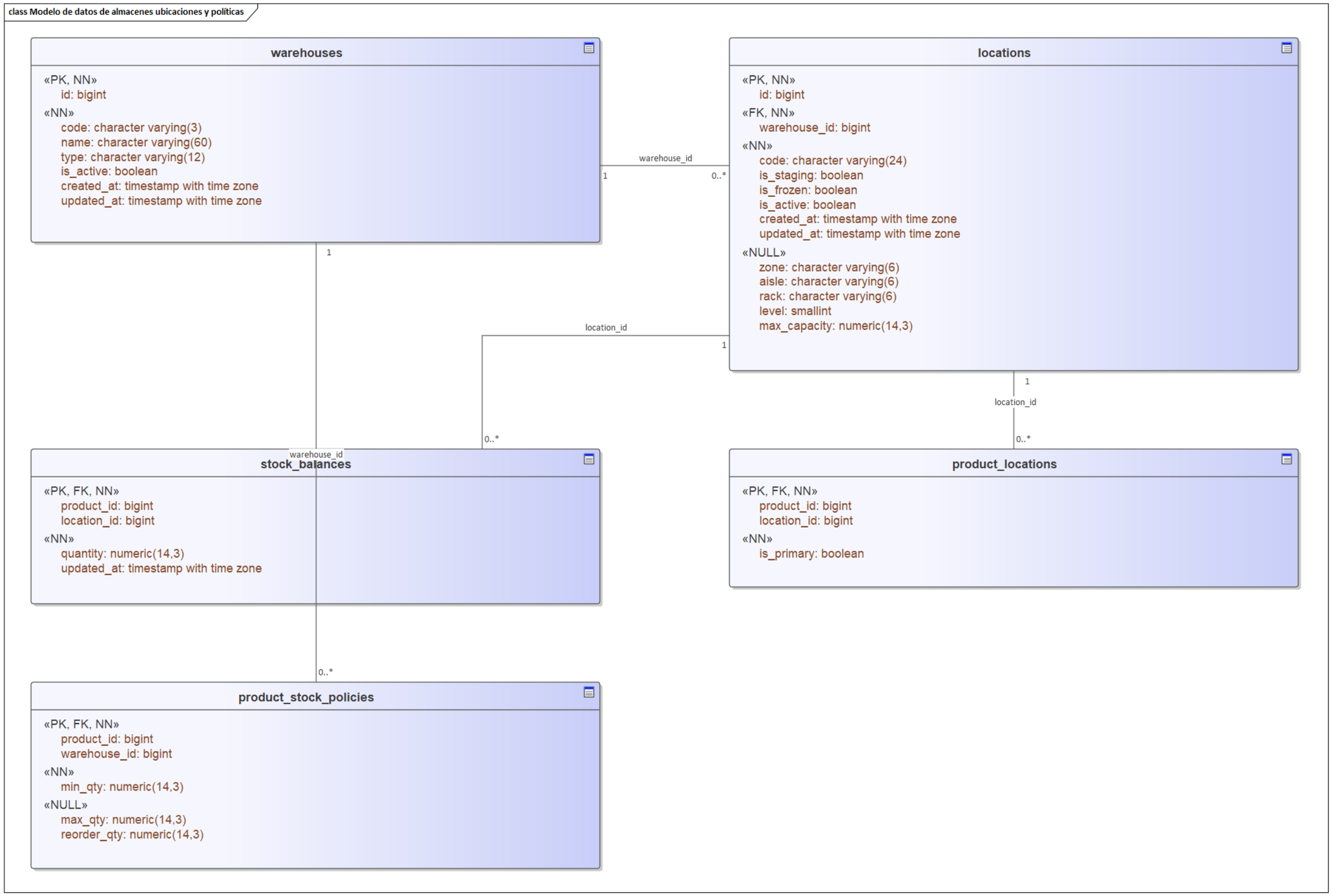
El ledger rechaza cambios destructivos mediante triggers. Las restricciones de saldo y las transacciones aportan controles adicionales a las validaciones PHP. Estas defensas se verificaron parcialmente contra wms_test. Las reglas propuestas de unicidad de apertura y conflicto de idempotencia necesitan reforzarse según los resultados del capítulo 6.

Figura 28 Modelo de datos de catálogo y códigos



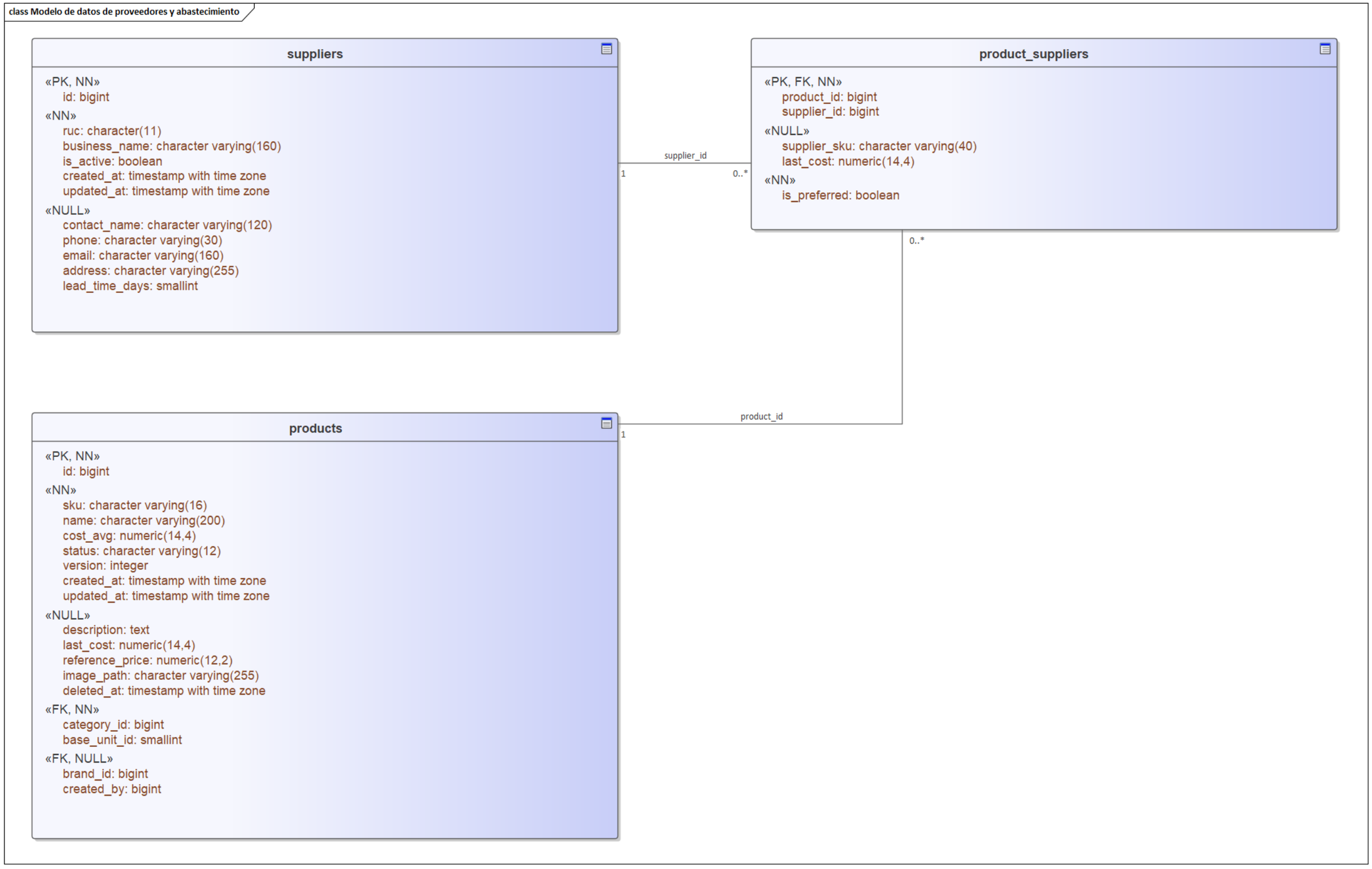
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 29 Modelo de datos de almacenes ubicaciones y políticas



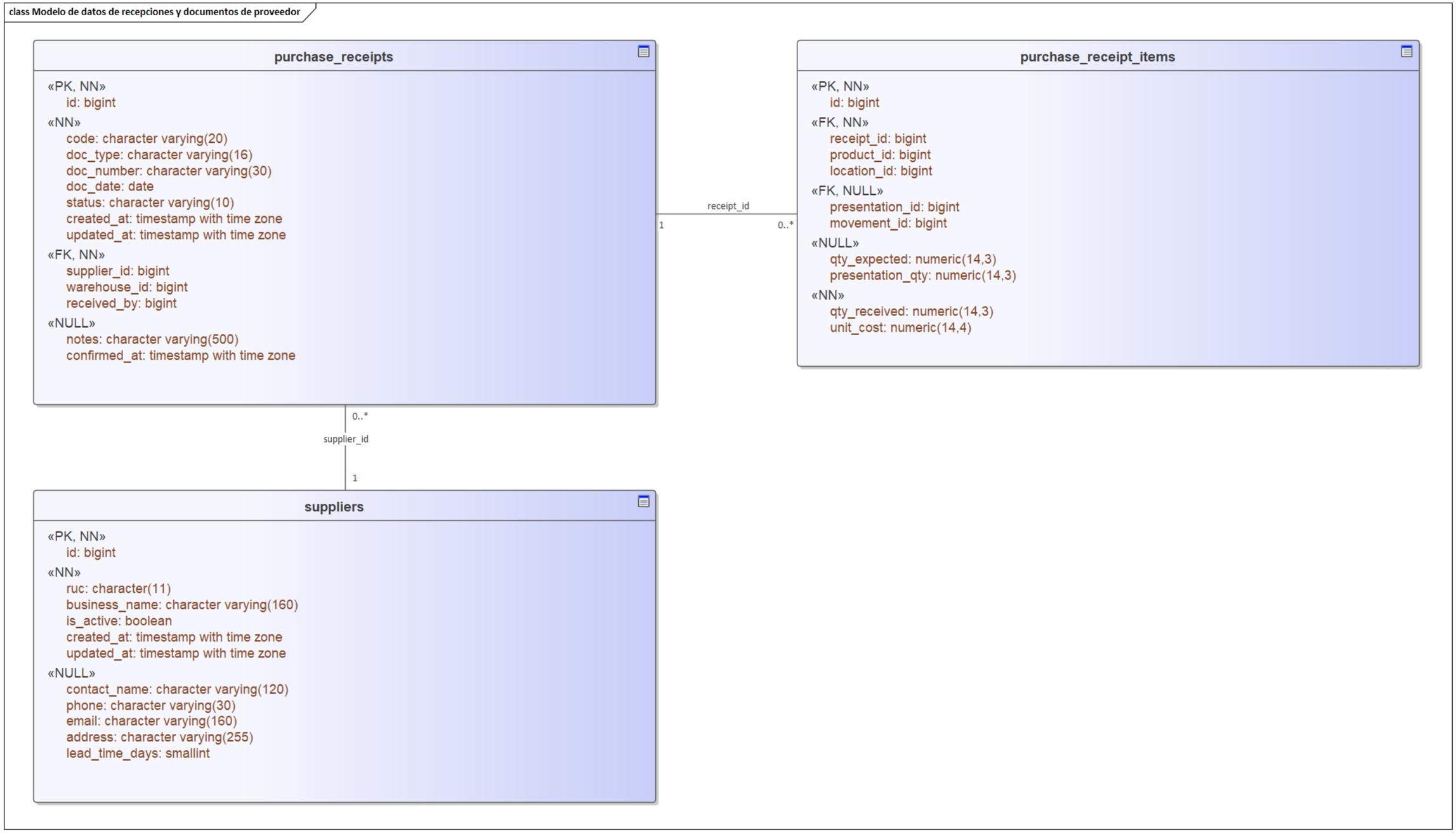
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 30 Modelo de datos de proveedores y abastecimiento



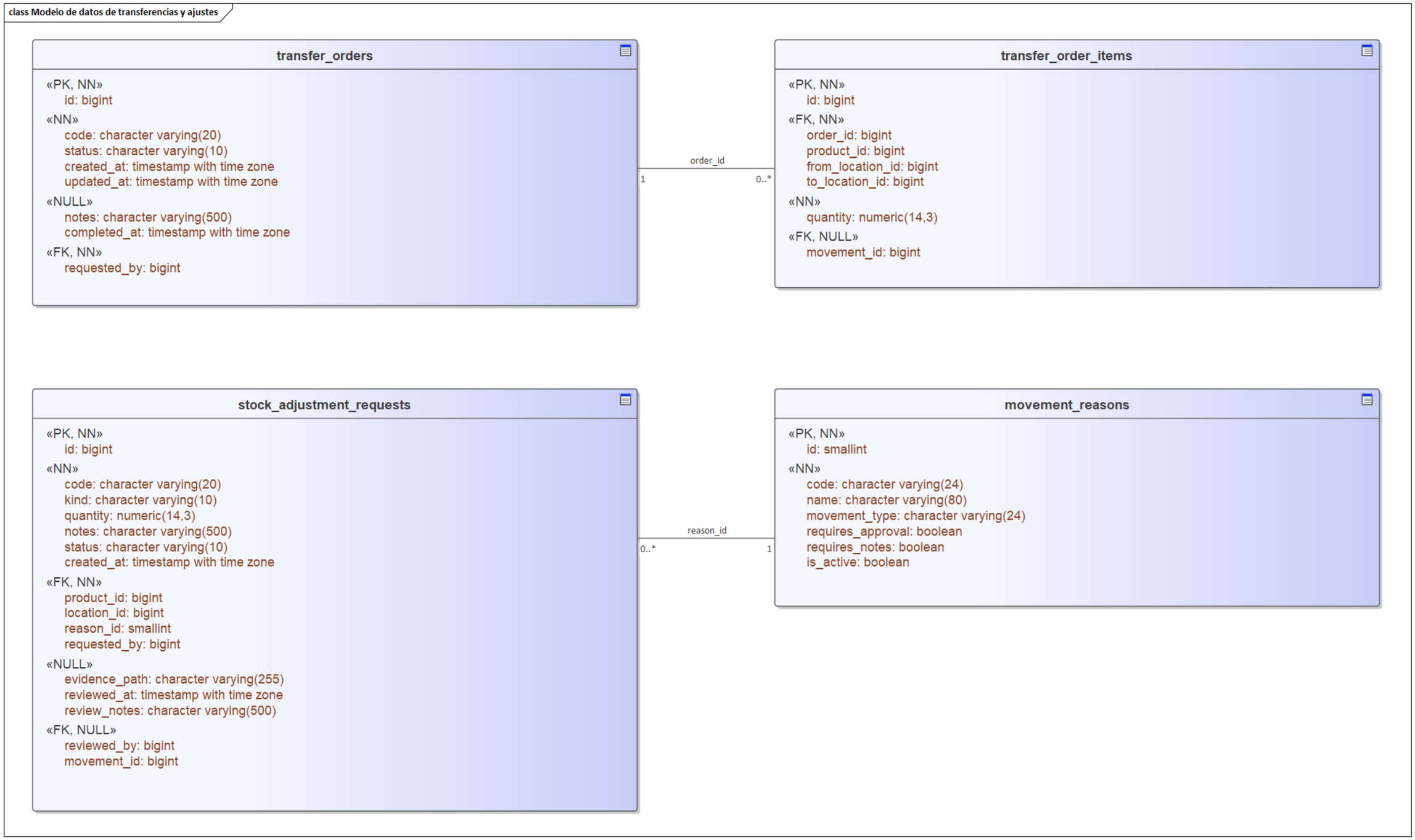
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 31 Modelo de datos de recepciones y documentos de proveedor



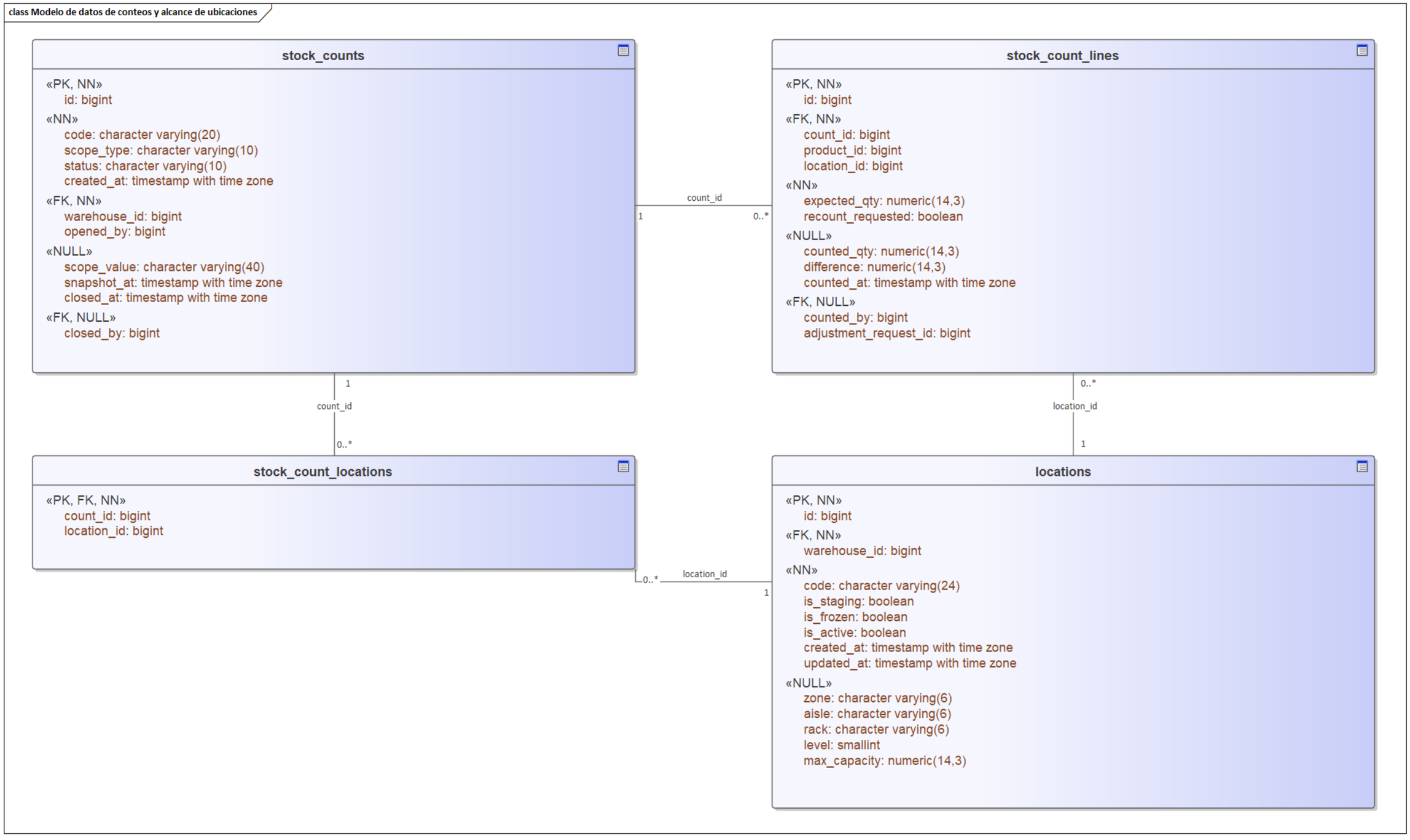
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 32 Modelo de datos de transferencias y ajustes



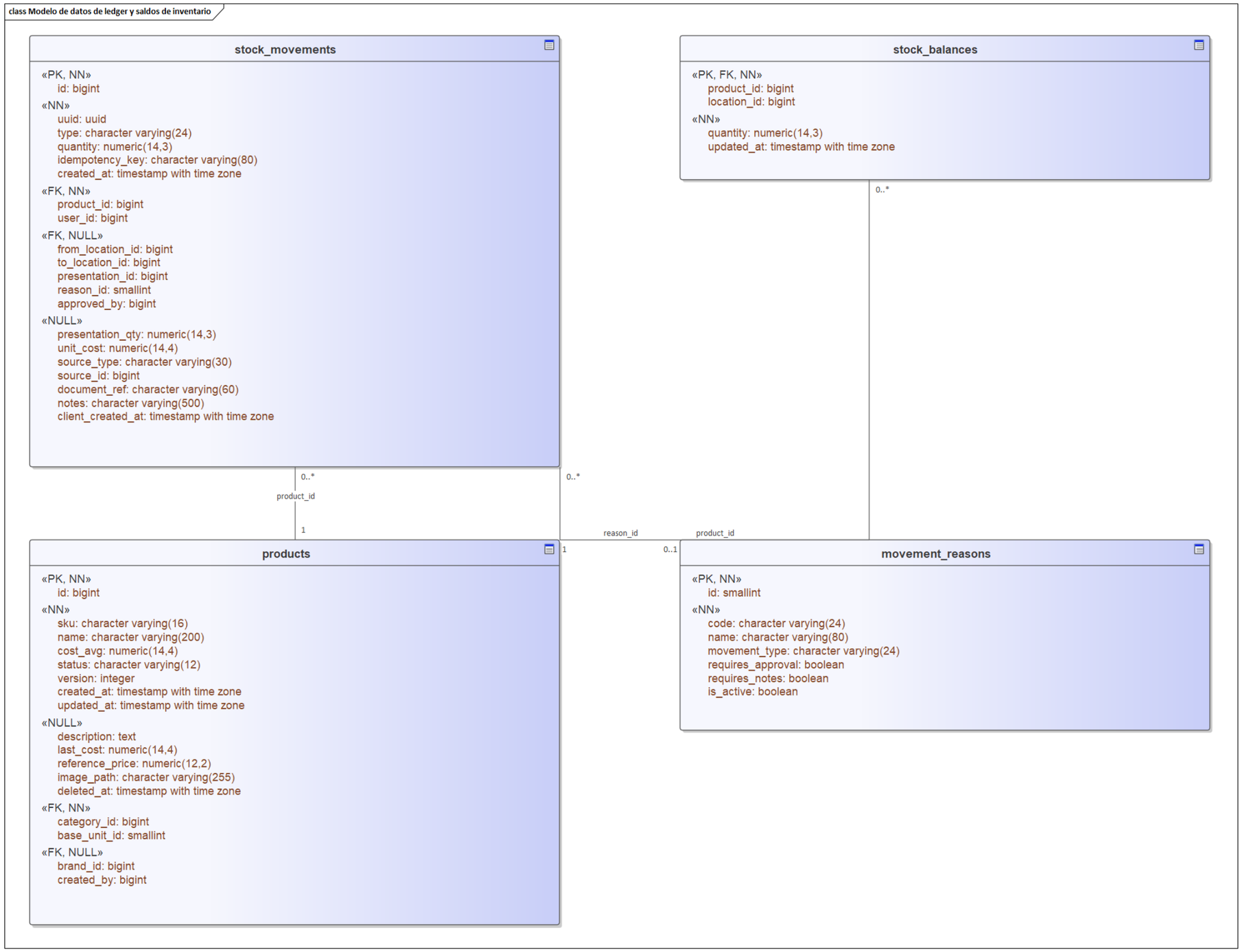
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 33 Modelo de datos de conteos y alcance de ubicaciones



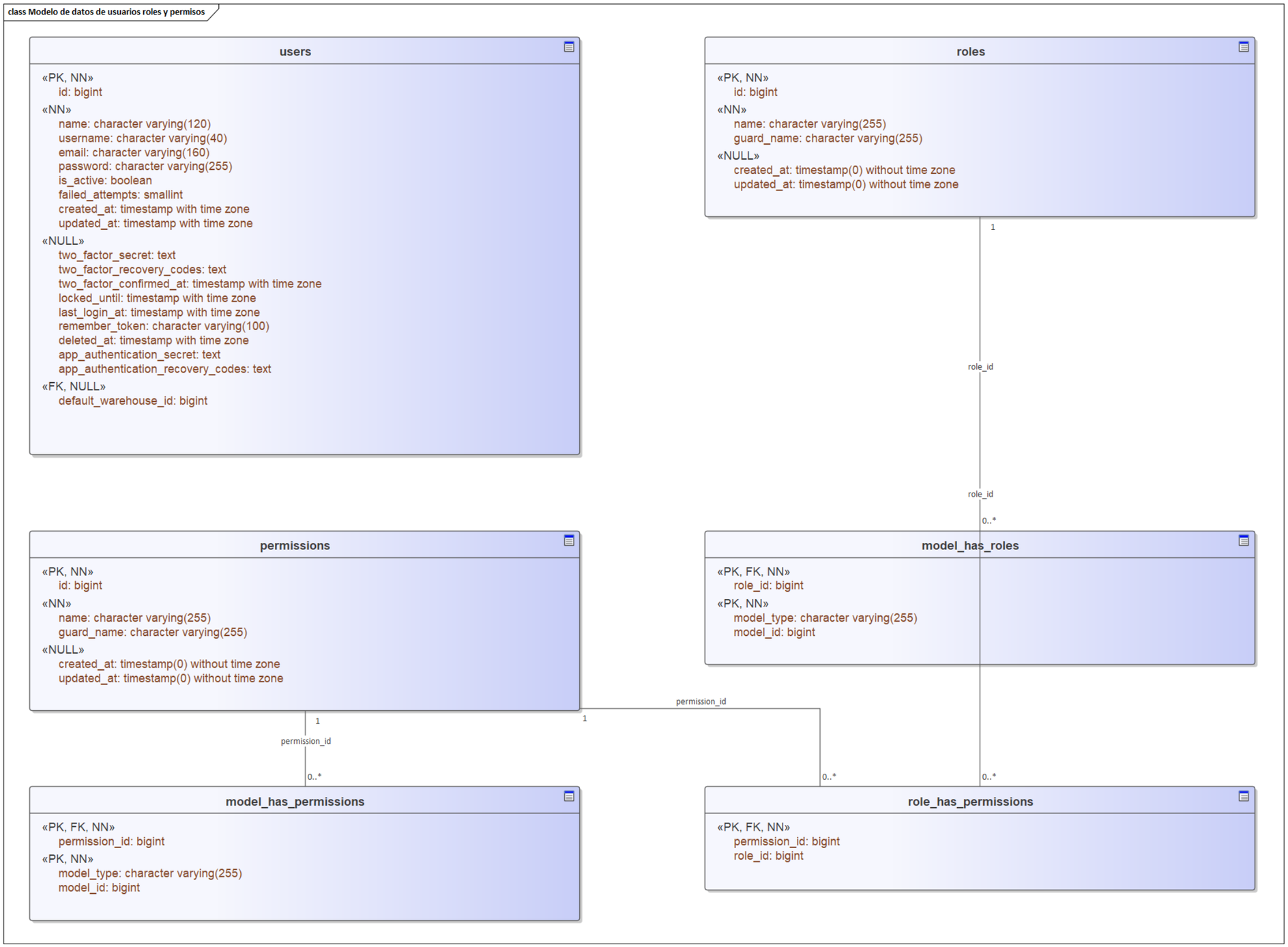
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 34 Modelo de datos de ledger y saldos de inventario



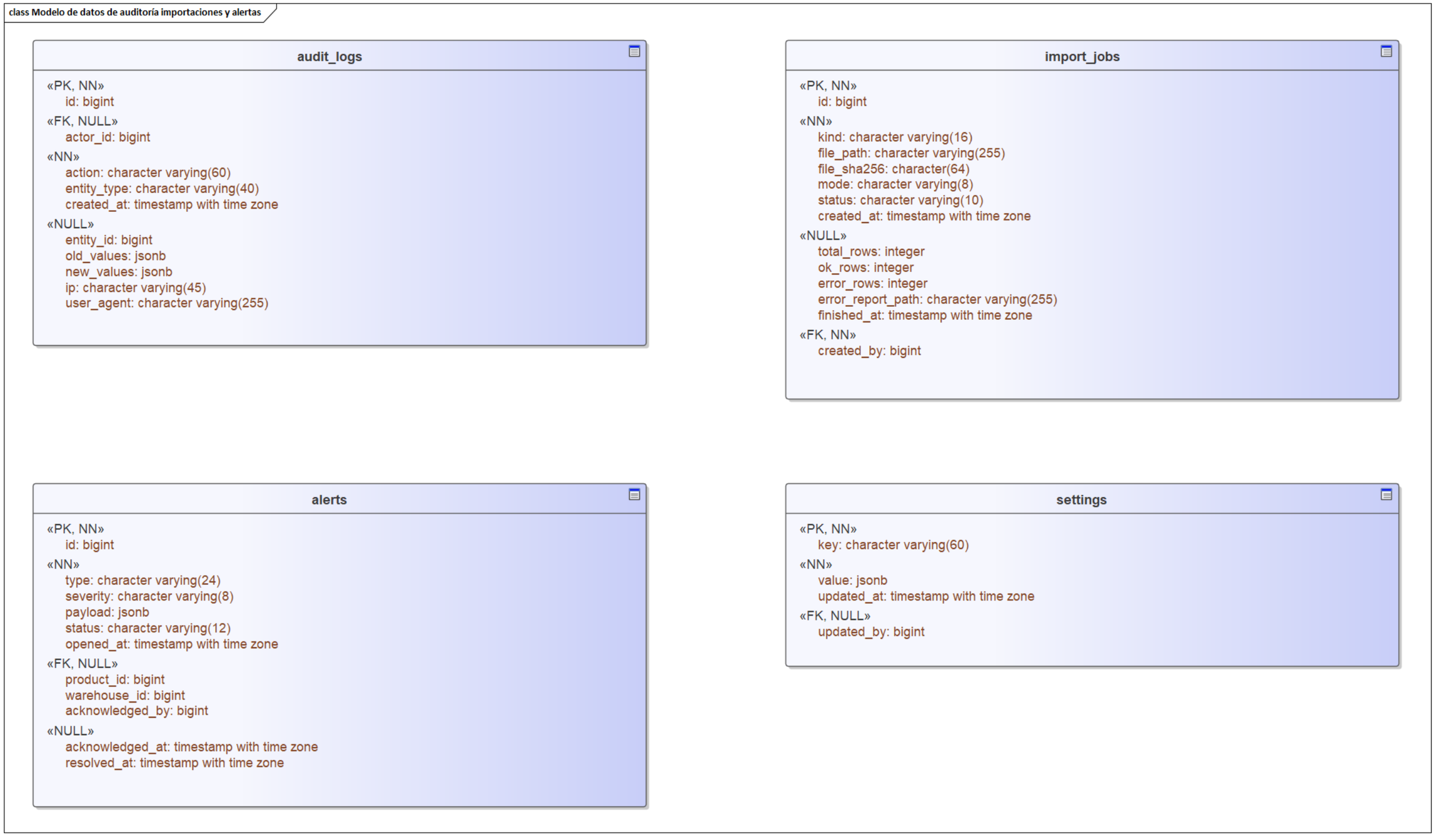
Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 35 Modelo de datos de usuarios roles y permisos



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 36 Modelo de datos de auditoría importaciones y alertas



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 37 Modelo de datos de soporte y persistencia operativa



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

Figura 38 Modelo de datos de infraestructura de colas y caché

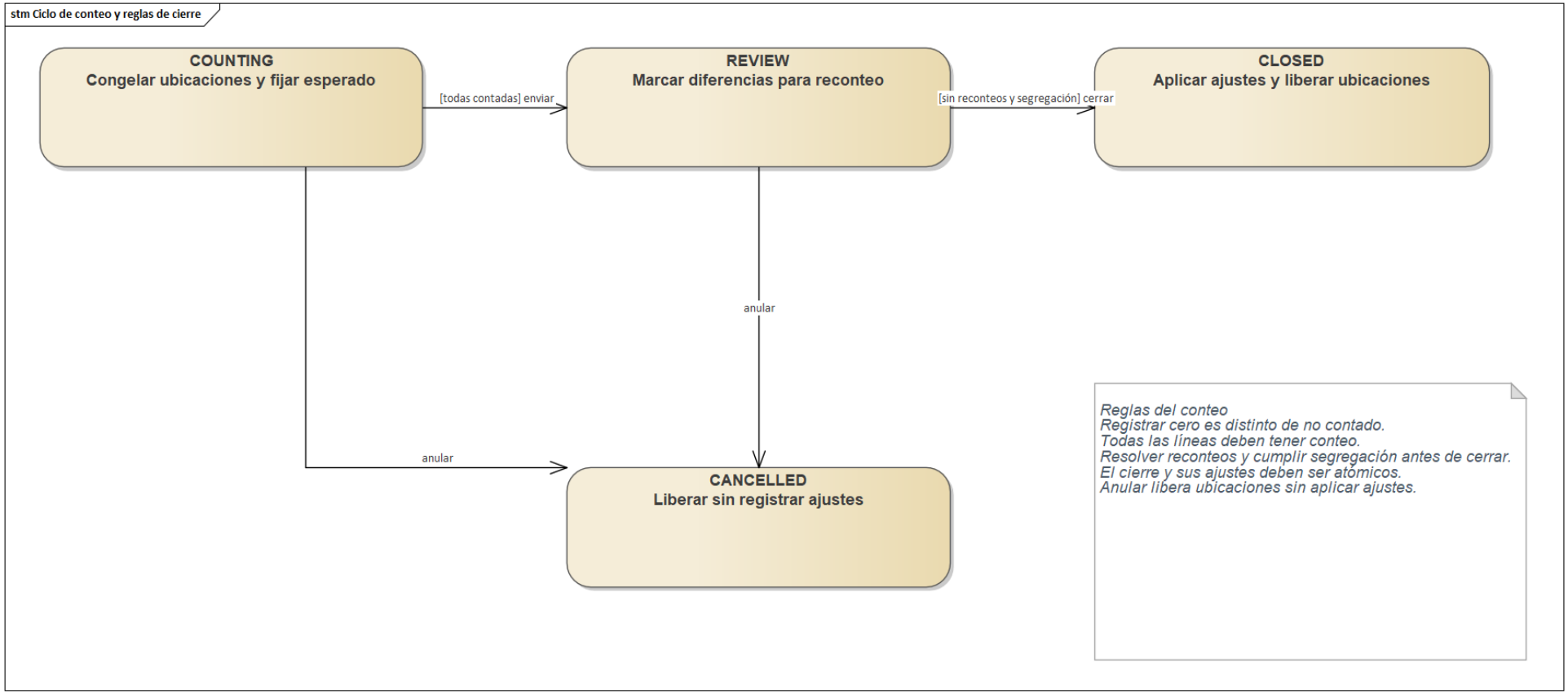


Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

5.2.3.1.4 Estados del conteo

El conteo distingue COUNTING, REVIEW, CLOSED y CANCELLED. Enviar a revisión exige que todas las líneas tengan valor contado; cero es distinto de no contado. El cierre exige resolver reconteos y cumplir la segregación de funciones cuando corresponde. La aplicación de ajustes y la liberación de ubicaciones deben ser atómicas. La anulación libera las ubicaciones sin registrar ajustes del cierre.

Figura 39 Ciclo de conteo y reglas de cierre



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

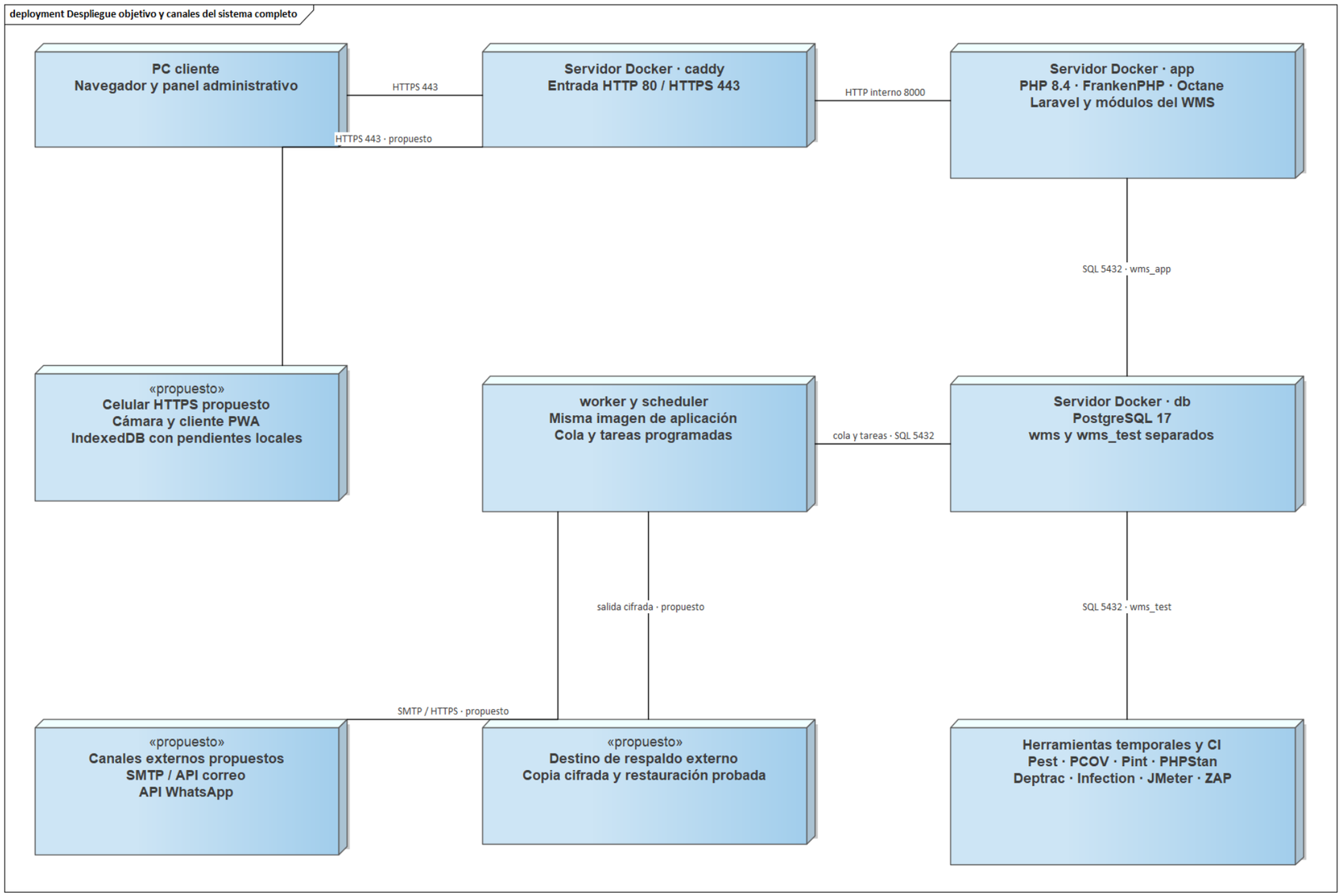
5.2.3.2 Arquitectura física

5.2.3.2.1 Diagrama de despliegue

Docker Compose define app, worker, scheduler, db, caddy y tools. Caddy recibe las conexiones en los puertos 80 y 443 y se comunica con la aplicación por el puerto interno 8000. PostgreSQL utiliza 5432. El servicio tools ejecuta los controles en contenedores temporales. Las conexiones de prueba deben seleccionar explícitamente wms_test y mantenerse separadas de wms.

La instancia en ejecución también exponía app por 8080; esa publicación no figura en la configuración Compose actual inspeccionada y debe reconciliarse en la documentación de instalación. La figura expresa el despliegue objetivo según Compose y distingue los canales móviles, externos y de respaldo todavía propuestos. La presencia del contenedor scheduler no demuestra que la conciliación nocturna esté programada: routes/console.php no contiene esa programación al corte.

Figura 40 Despliegue objetivo y canales del sistema completo



Fuente: elaboración a partir del backlog completo y del código del proyecto, mediante Enterprise Architect. «propuesto» identifica diseño pendiente de implementación.

CAPÍTULO 6 ARTEFACTOS DE PRUEBA

6.1 Casos de prueba

6.1.1 Pruebas unitarias

La suite incluye validación de valores y reglas de catálogo, usuarios, ubicaciones, proveedores, inventario, operaciones y auditoría. Una comprobación adicional cargó las clases reales y utilizó dobles en memoria para ejecutar 41 criterios específicos con sus entradas documentadas. Cuarenta resultaron conformes y uno detectó el conflicto de idempotencia. Los dobles aíslan el dominio; no acreditan persistencia ni criptografía real cuando se sustituye el hasher.

6.1.2 Pruebas de integración

Las pruebas de integración utilizan Laravel y PostgreSQL. La comprobación adicional de cinco escenarios se ejecutó con APP_ENV=testing y DB_DATABASE=wms_test, después de comprobar el nombre efectivo de la base. Sus inserciones se realizaron dentro de una transacción revertida al finalizar. Se verificaron las protecciones del ledger, la idempotencia, la apertura y la visibilidad de la acción de exportación. El esquema de prueba fue migrado de manera explícita sin ejecutar una reconstrucción destructiva de la base operativa.

6.1.3 Pruebas de sistemas

La suite contiene pruebas de recursos, páginas y acciones del panel mediante las herramientas del framework. Esto aporta evidencia de integración de componentes, pero no acredita todos los flujos de navegador, compatibilidad de dispositivos, accesibilidad ni aceptación en la empresa. Las pruebas completas de sistema, cámara y sincronización quedan pendientes según su estado en el backlog.

6.1.4 Pruebas de aceptación

Se especificaron 191 criterios para las 63 historias. Al corte existen 45 criterios distintos ejecutados mediante las comprobaciones adicionales: 42 conformes y tres no conformes. Hay 119 casos con código o pruebas relacionadas que aún no se ejecutaron con la entrada exacta del nuevo criterio y 27 vinculados con capacidades pendientes de implementar. La prueba de idempotencia aparece en ambos niveles y se cuenta una sola vez en el total de criterios distintos.

Criterio	Escenario y entrada	Resultado observado
CA-017-2	Intentar UPDATE sobre el ledger	PostgreSQL rechaza la modificación
CA-017-3	Intentar DELETE y TRUNCATE CASCADE	PostgreSQL rechaza las operaciones
CA-019-4	Repetir UUID y clave cambiando cantidad de 10 a 99	Se devuelve duplicado sin advertir el contenido distinto; no conforme
CA-053-2	Registrar dos aperturas de 10 con identificadores distintos	Se obtiene saldo 20,000; no conforme con apertura única

Criterio	Escenario y entrada	Resultado observado
CA-045-4	Rol lectura sin reports.export consulta acción de reposición	La acción aparece visible; no conforme con el permiso esperado

Los 36 criterios iniciales del Excel se conservaron y se relacionaron con evidencia de dominio o integración según correspondía. La ejecución técnica no constituye aprobación del usuario final. Las entradas, pasos, responsables técnicos y resultados de cada escenario se consultan en la hoja Criterios de Aceptación; los resultados JSON permiten contrastar los casos ejecutados.

6.2 Resultados de pruebas

6.2.1 Cobertura de código

6.2.1.1 Cobertura de declaraciones

La ejecución instrumentada con PCOV registró 1913 líneas ejecutables cubiertas de 1997 en app/Modules, equivalentes a 95,79 por ciento. También se registraron 320 de 370 métodos cubiertos y 99 de 138 clases cubiertas, según el criterio del generador. El filtro comprende 169 archivos de módulos y excluye el panel, los modelos y las políticas situados fuera de esa ruta; el resultado no se presenta como cobertura de toda la aplicación.

6.2.1.2 Cobertura de ramas

PCOV no proporcionó medición de ramas para esta ejecución. El valor cero de condicionales en el archivo de cobertura significa ausencia de esa medición, no cobertura de ramas igual a cero ni cobertura completa. Se requiere instrumentación compatible y una ejecución específica para informar el indicador.

6.2.1.3 Cobertura de caminos

No se realizó una medición de caminos. Las combinaciones de escenarios funcionales y las pruebas de frontera complementan la validación, pero no sustituyen una métrica instrumentada de caminos ni prueban todas las combinaciones posibles de ejecución.

6.2.2 Tasa de éxito en pruebas

La ejecución ordinaria de Pest obtuvo 253 pruebas aprobadas, 1363 aserciones y ninguna prueba fallida u omitida. Su duración fue de 365,80 segundos. La ejecución con cobertura repitió 253 aprobadas y 1363 aserciones en 353,51 segundos. Estas duraciones corresponden a la suite y no constituyen tiempos de respuesta de la aplicación.

La tasa de éxito de la suite existente es 253 de 253, equivalente a 100 por ciento. Para la aceptación adicional, la tasa es 42 de 45, equivalente a 93,33 por ciento de los criterios distintos ejecutados. Los 146 criterios sin ejecución específica se informan por separado y no se excluyen para declarar aceptado el producto completo. La proporción de criterios conformes sobre el total especificado es 42 de 191; este indicador mide verificación documentada y no avance físico del software.

6.3 Métricas de calidad

6.3.1 Métricas estáticas

Control	Resultado observado	Alcance y límite
Laravel Pint	377 archivos conformes	Formato del código incluido en la ejecución
PHPStan nivel 6	Sin errores	Reglas y rutas de la configuración actual
Deptrac	Cero infracciones y cero no cubiertas; 769 permitidas	app/Modules y app/Shared; no cubre toda la presentación real
Composer validate	Validación sintáctica satisfactoria	No demuestra instalación limpia ni resuelve dependencias omitidas

La configuración de Deptrac busca una capa Presentation bajo los módulos, mientras la interfaz real reside en app/Filament. Las pruebas de arquitectura enumeran explícitamente Inventory, Catalog, Iam y Audit. Se recomienda ampliar el alcance a los demás módulos y a las dependencias efectivas del panel antes de presentar el análisis como verificación arquitectónica total.

6.3.2 Métricas dinámicas

Se dispone de pruebas funcionales e integración y de cobertura de líneas. No se midieron latencia de operaciones con carga, consumo de recursos bajo concurrencia, puntuación SUS, recuperación de respaldos ni seguridad dinámica. Esas mediciones requieren un protocolo con datos, usuarios, entorno, volumen y criterios de salida definidos.

6.3.3 Análisis de resultados y conclusiones

Los controles automáticos respaldan una base técnica consistente dentro de su alcance. Las comprobaciones adicionales evidencian que una suite aprobada puede omitir escenarios relevantes. D01 exige comparar el contenido de un reintento con el movimiento original; D02 exige definir y hacer cumplir la unicidad de la apertura; D03 exige revisar la autorización de la exportación. Estos criterios añadidos deben ratificarse con negocio y convertirse en pruebas de regresión permanentes.

El kardex limita la consulta a 500 movimientos sin paginación ni aviso de truncamiento. Las exportaciones revisadas utilizan vistas globales y no trasladan automáticamente los filtros de la tabla. La exactitud de conteo observada trunca 2 de 3 a 66,66 por ciento; la política de presentación debe acordarse. Estos puntos requieren especificación y validación, sin atribuir por defecto un resultado de negocio que el plan no había definido.

6.4 Aseguramiento de la calidad en el proyecto

6.4.1 Aplicación de ISO 25010 en el aseguramiento de la calidad

La matriz vincula requisitos con pruebas y resultados, mientras el plan de calidad relaciona atributos con métricas. La revisión del segundo avance distingue evidencia ejecutada, diseño pendiente y aceptación empresarial. Se deberán incorporar mediciones de operación y usuarios para evaluar el efecto del sistema en el contexto de la empresa.

6.4.2 Evaluación de atributos del sistema

La funcionalidad cuenta con evidencia automatizada y tres brechas específicas abiertas. La fiabilidad dispone de transacciones y restricciones probadas parcialmente, pero falta verificar restauración. La mantenibilidad cuenta con controles estáticos y cobertura del ámbito modular; la instalación limpia requiere declarar dependencias faltantes. Rendimiento, interacción con usuarios y adaptación móvil permanecen sin evaluación final. Esta evaluación parcial no acredita certificación ISO ni preparación para producción.

CAPÍTULO 7 MANUALES Y CÓDIGO FUENTE

7.1 Manual de usuario

7.1.1 Guía de instalación

El punto de partida verificable es un entorno con Docker Engine y Docker Compose, los servicios de la configuración del proyecto y las variables de conexión correspondientes. Antes de entregar una guía final deben corregirse las dependencias no declaradas, ensayar una instalación limpia y reconciliar los puertos de la instancia con Compose. Las credenciales y datos sensibles no se incorporan al informe.

7.1.2 Guía de uso

El flujo operativo previsto es acceder con un rol autorizado, mantener el catálogo, preparar ubicaciones, registrar operaciones y consultar saldos, kardex y reportes. Los conteos requieren registrar cada línea, revisar diferencias y cerrar con las guardas establecidas. El manual ilustrado con pasos de navegador y usuarios de la empresa se completará después de validar los permisos y resolver los escenarios no conformes.

7.1.3 Resolución de problemas comunes

Ante una operación rechazada debe comprobarse el rol, el estado del documento, la disponibilidad y las reglas del producto. No se debe repetir una apertura para corregir un saldo: las correcciones requieren un procedimiento autorizado y trazable. La instalación y la recuperación se ensayarán en un entorno separado antes de intervenir datos operativos.

7.2 Código fuente

El proyecto se organiza en app/Modules, app/Filament, app/Models, app/Policies, database, tests y archivos de configuración de Docker y calidad. Se entregan las evidencias de esta revisión junto con los modelos; no se declara una publicación remota ni un despliegue productivo como realizados. La documentación final deberá fijar la versión entregada y las instrucciones reproducibles del repositorio.

CONCLUSIONES Y RECOMENDACIONES

El segundo avance dispone de una arquitectura modular y de una suite que aprobó 253 pruebas y 1363 aserciones. La cobertura de líneas del ámbito app/Modules supera la meta del plan, aunque no mide interfaz, ramas ni caminos. Las tres no conformidades de aceptación y la instalación no reproducible impiden considerar cerrado el aseguramiento de calidad.

El diseño documenta todo el producto previsto en el backlog, incluidas las capacidades futuras. Su separación respecto de los resultados de ejecución evita confundir una propuesta con una función disponible. El Excel concentra la trazabilidad de los 191 criterios y constituye el registro que deberá actualizarse con cada nueva validación.

Se recomienda resolver y revalidar D01 a D03, declarar Filament y Livewire en las dependencias, ratificar roles y reglas de apertura con la empresa, completar importadores y automatizaciones y ejecutar la validación completa de interfaz. Después deberán medirse rendimiento, SUS y recuperación, y registrar las revisiones y la aceptación empresarial antes del cierre.

BIBLIOGRAFÍA

International Organization for Standardization. (2023). ISO/IEC 25010:2023 Systems and software engineering Product quality model. https://www.iso.org/standard/78176.html

Object Management Group. (2017). Unified Modeling Language Version 2.5.1. https://www.omg.org/spec/UML/2.5.1/PDF

Schwaber, K., y Sutherland, J. (2020). La Guía Scrum. https://scrumguides.org/scrum-guide.html

Sparx Systems. (s. f.). UML tutorials. https://sparxsystems.com/resources/tutorials/uml-tutorials.html

Laravel. (2026). Laravel 13 documentation. https://laravel.com/framework/docs/13.x

Filament. (s. f.). Filament 5 documentation. https://filamentphp.com/docs/5.x/introduction/overview

Pest. (s. f.). Test coverage. https://pestphp.com/docs/test-coverage

PostgreSQL Global Development Group. (s. f.). PostgreSQL 17 Constraints. https://www.postgresql.org/docs/17/ddl-constraints.html

Equipo del proyecto. (2026). Informe académico original y Product Backlog WMS Fase Inicial Cronograma. Documentos internos del proyecto.

Plan de prácticas preprofesionales PMI Scrum mejorado y Guía docente del plan de prácticas. (s. f.). Materiales de referencia proporcionados para organizar la gestión y las evidencias del proyecto.

ANEXOS

El libro Product_Backlog_WMS_Segundo_Avance.xlsx contiene Backlog del Producto, Sprints, Cronograma, Tareas, Sprint 1, Criterios de Aceptación, Indicaciones, Ejemplo y Gestión PMI. Los escenarios de Ejemplo son material orientativo y no resultados del sistema.

El paquete de modelos contiene el repositorio editable Diseno_Integral_WMS.eap y las figuras por área. El diccionario de datos conserva las 46 tablas físicas, sus columnas y las 66 referencias verificadas. El archivo ERwin contiene el inventario de entidades, atributos y claves primarias. Las relaciones completas se consultan en Enterprise Architect y en el diccionario de datos.

El paquete de evidencia conserva los resultados JUnit y de cobertura, las salidas de Pint, PHPStan y Deptrac y los resultados de las comprobaciones adicionales. El Mini Informe de Validación detalla el procedimiento y las limitaciones de los hallazgos. Estos anexos permiten reproducir y ampliar la verificación sin incluir credenciales ni datos operativos de la empresa.